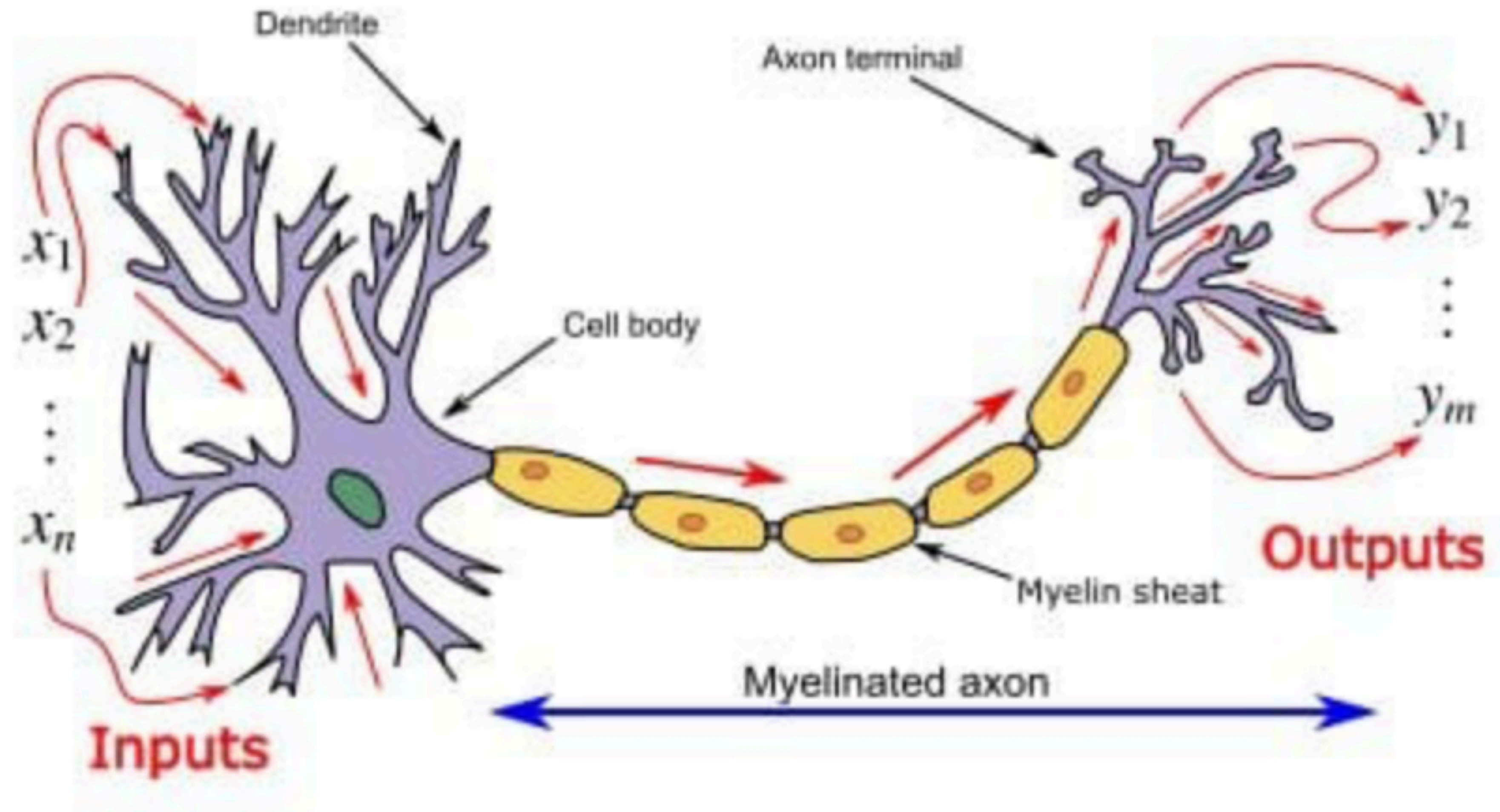


Bazele Inteligenței Artificiale

Lab 8. Rețele neuronale artificiale și perceptroni multistrat

Efrem Dragoș-Sebastian-Mihaly

Cum procesează neuronii biologici informație?



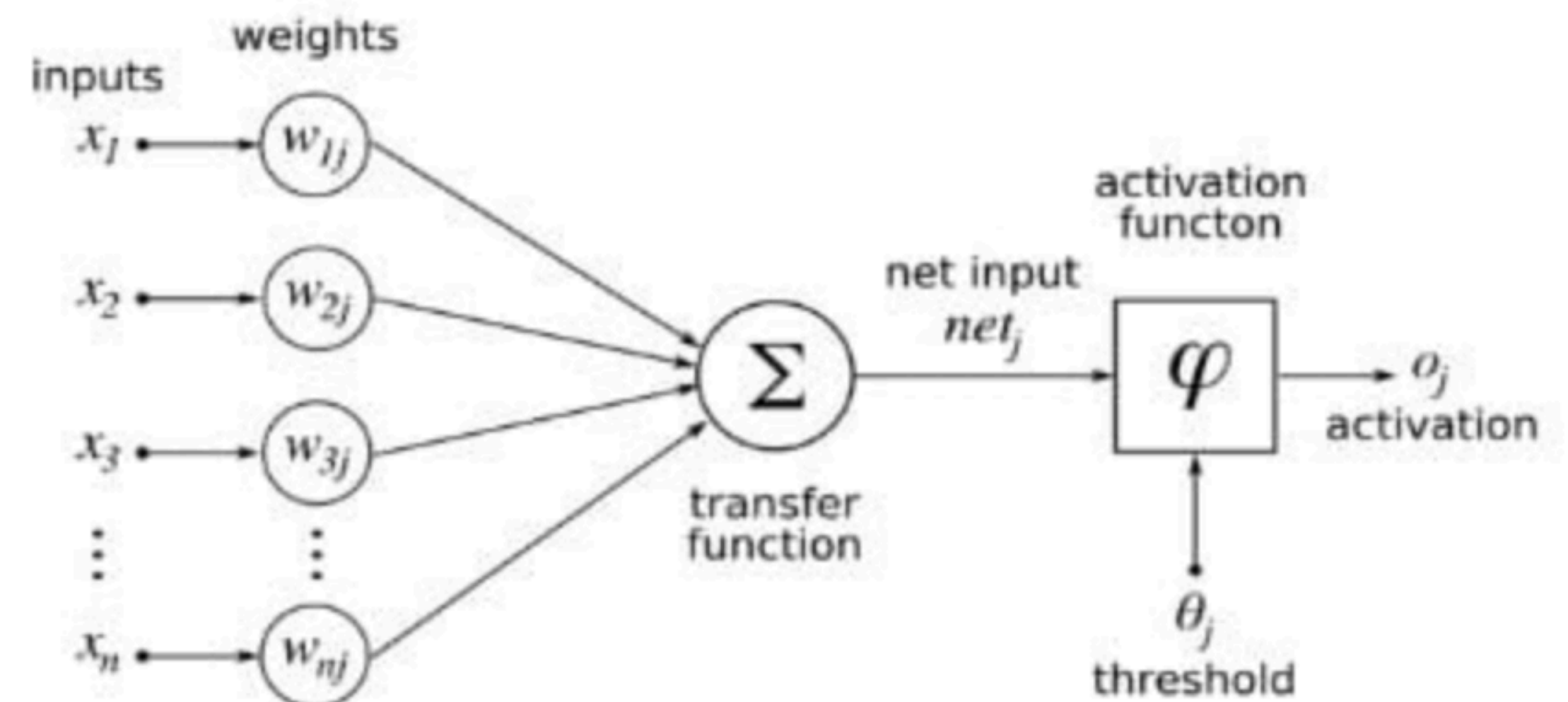
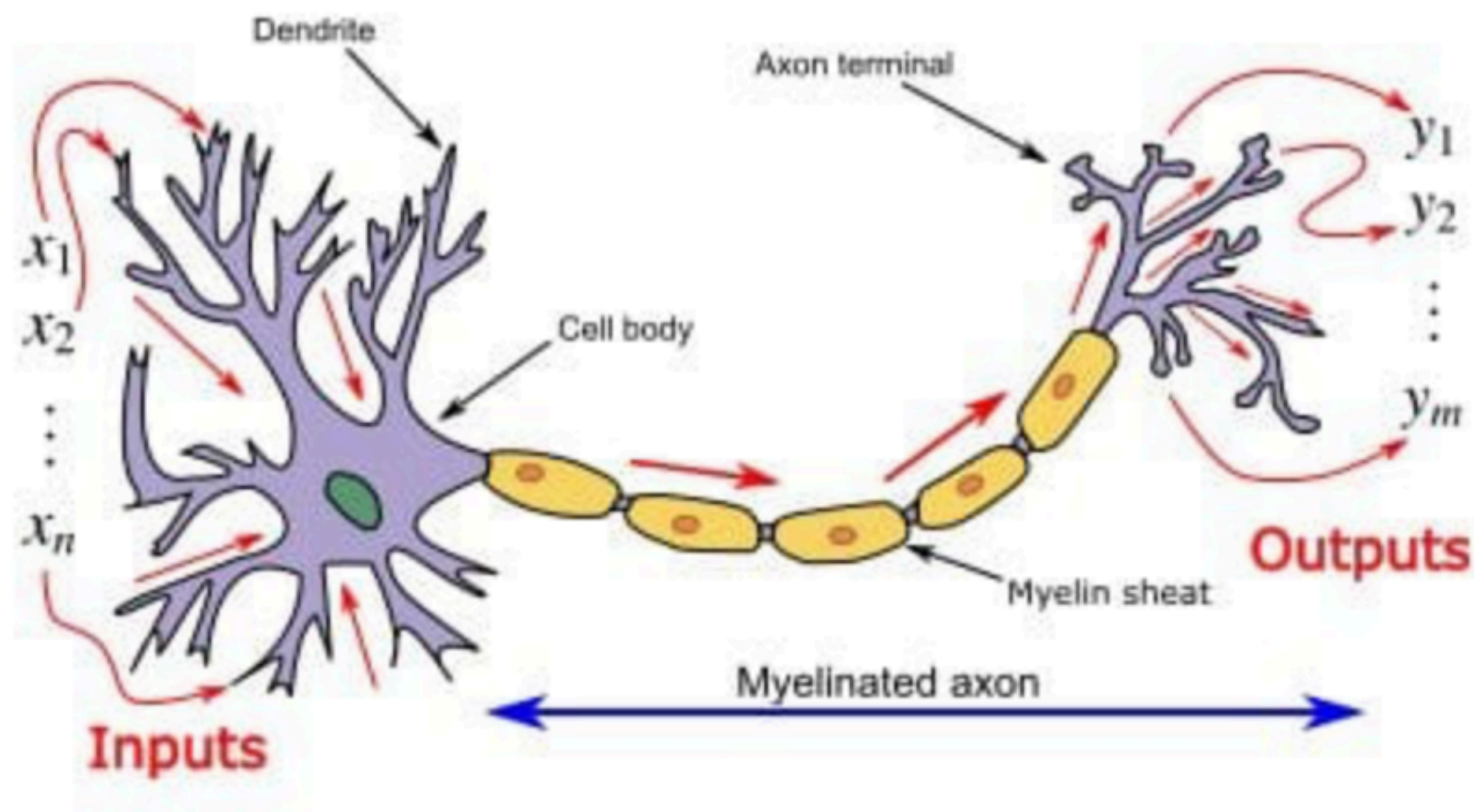
Cum procesează neuronii biologici informație?

- Dendritele (Intrările):
 - Sunt extensii ramificate care primesc semnale de la alți neuroni sau stimuli externi $(x_1, x_2, \dots, x_n)$
 - Fiecare semnal de intrare vine sub formă de impuls electric sau chimic (mai slab sau mai puternic), care este colectat de dendrite și transmis către corpul celular
- Corpul celular (Procesatorul):
 - Aici se integrează toate semnalele primite din dendrite
 - Decide dacă semnalele colectate sunt suficient de puternice pentru a fi transmise mai departe (adică, dacă depășesc un anumit “prag”)

Cum procesează neuronii biologici informație?

- Axonul:
 - Este un “cablu” lung care transmite semnalul de la corpul celular către alte celule (neuroni, mușchi etc.)
 - Este învelit într-un strat protector numit teaca de mielină. Aceasta accelerează transmiterea semnalelor
- Ieșiri:
 - La capătul axonului, semnalul ajunge la alte celule prin sinapse (conexiuni neuronale)
 - Semnalul poate fi transmis la diferiți neuroni simultan

Aproximarea rețelelor neuronale biologice



Aproximarea rețelelor neuronale biologice

- Intrări ponderate:
 - Fiecare intrare x_i este înmulțit cu o pondere w_{ij} care reflectă importanța acestuia pentru neuron
 - Aceasta imită modul în care conexiunile sinaptice dintr-un neuron biologic sunt mai slabe sau mai puternice
- Funcția de transfer (Σ):
 - Toate semnalele ponderate se adună într-un proces matematic (sumă ponderată)
 - În biologie, acest proces corespunde acumulării semnalelor în corpul celular

Aproximarea rețelelor neuronale biologice

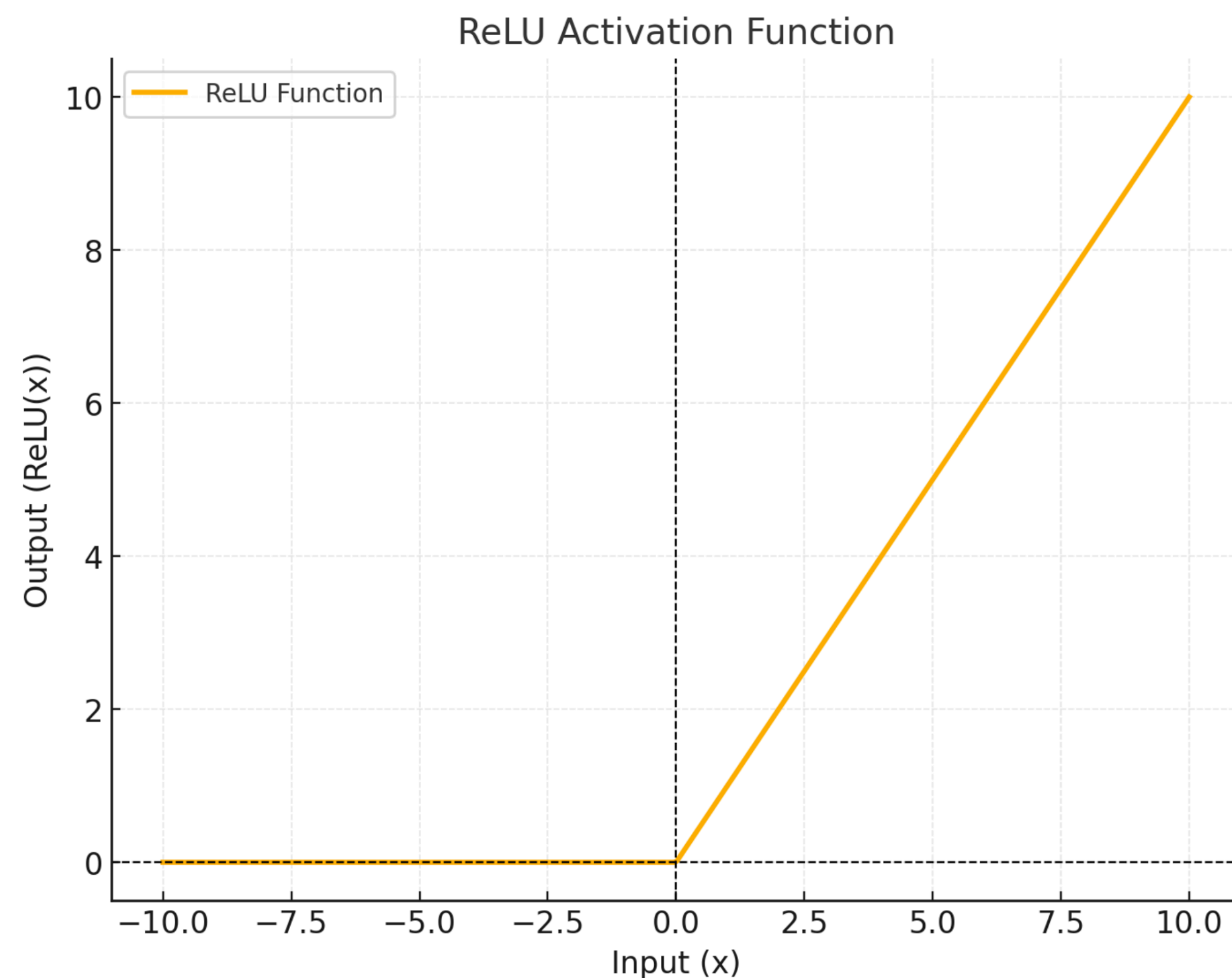
- Funcția de activare (ϕ):
 - După ce semnalul total este calculat, neuronul decide dacă să se “activeze” sau nu, aplicând o funcție de activare
 - În biologie, aceasta reflectă decizia corpului celular de a genera sau nu un impuls
- Ieșiri:
 - Rezultatul final al neuronului (activat sau nu) este transmis următoarelor noduri din rețea.
 - În biologie, aceasta este echivalentă cu impulsurile care călătoresc pe axon spre următorii neuroni.

Funcția de activare

- Neuronul artificial primește mai multe valori de intrare $x_1, x_2, \dots, x_n$, fiecare ponderată cu o valoare $w_1, w_2, \dots, w_n$
- Toate acestea sunt adunate într-un singur număr $\sum (x_i \cdot w_i) + b$ unde b este numit bias
- Acest număr prezintă o relație liniară între variabile care poate sau nu să fie utilă:
 - În prelucrarea imaginilor, obiectele pot avea poziții, dimensiuni și forme diferite, adică o relație non-liniară
 - În clasificare, frontierele de separare dintre clase nu sunt întotdeauna drepte, ele pot fi curbe sau forme complexe
 - Modele precum cele de prognoză meteo sau piețe financiare implică relații non-liniare între variabile

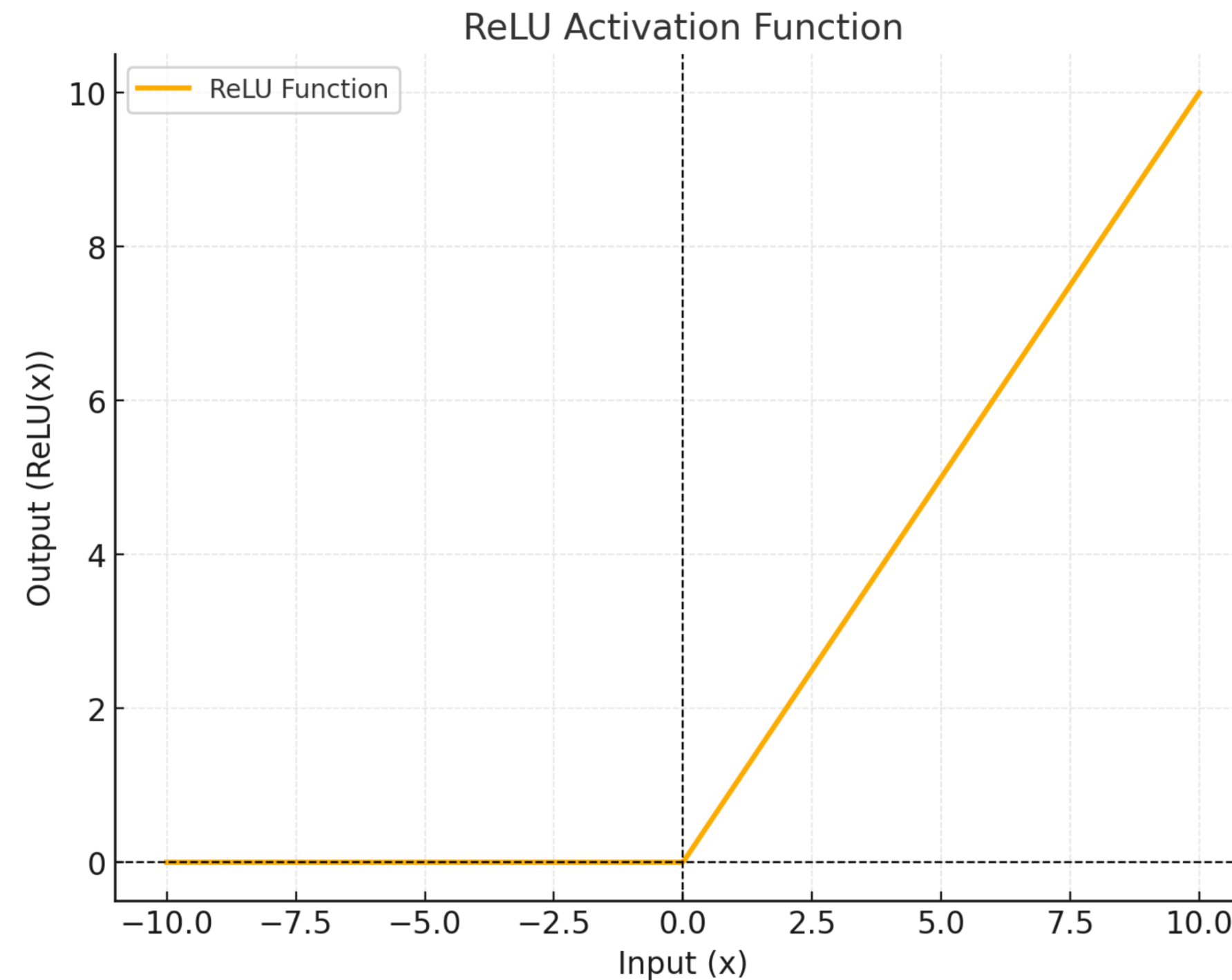
Funcția de activare

- Scopul funcției de activare este să facă semnalul non-liniar (dacă e nevoie) sau să “taie” valorile care nu contează
- Spre exemplu, haideți să vedem funcția de activare ReLU:



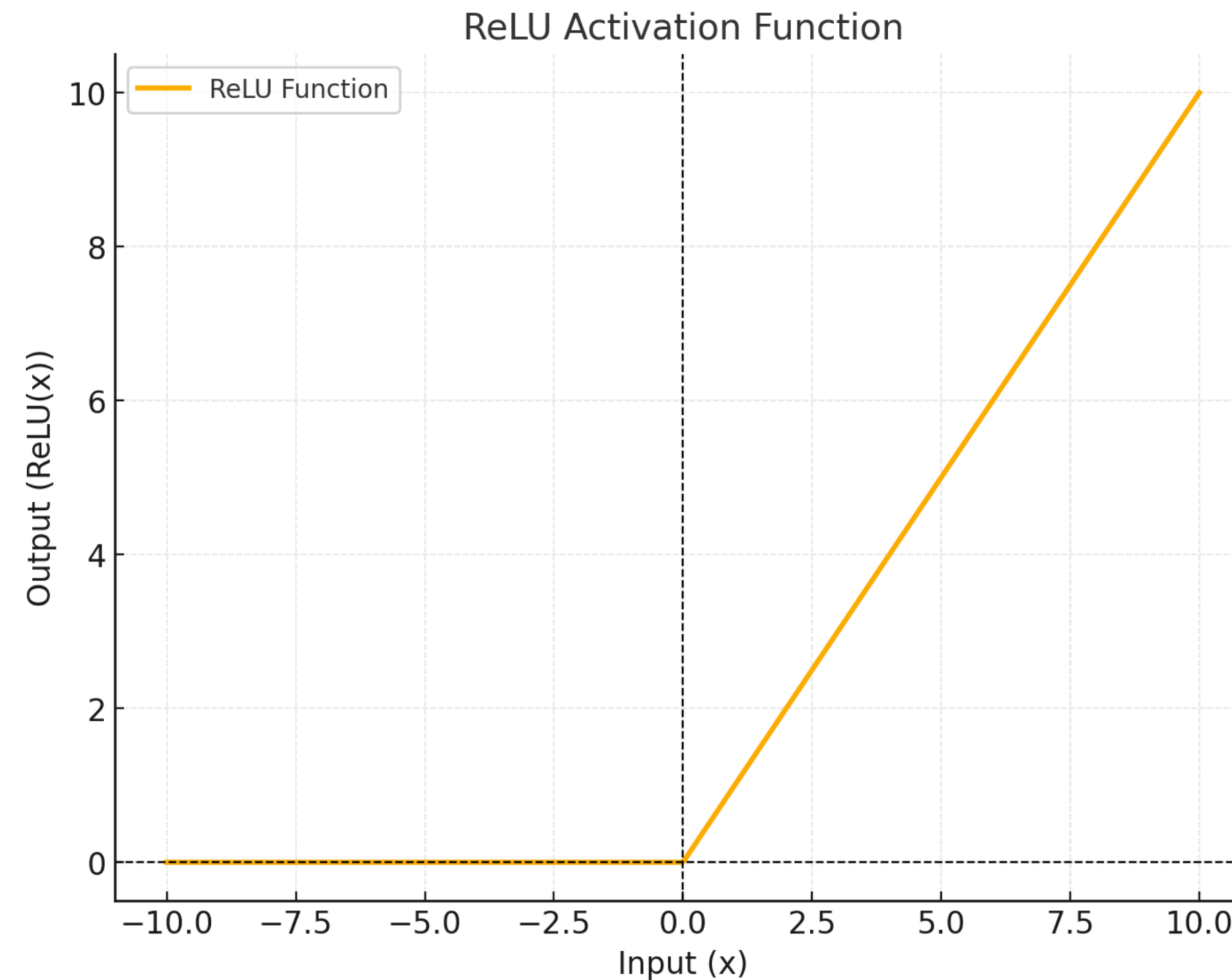
$$ReLU(x) = \begin{cases} x, & \text{if } x > 0 \\ 0, & \text{if } x \leq 0 \end{cases}$$

Funcția de activare



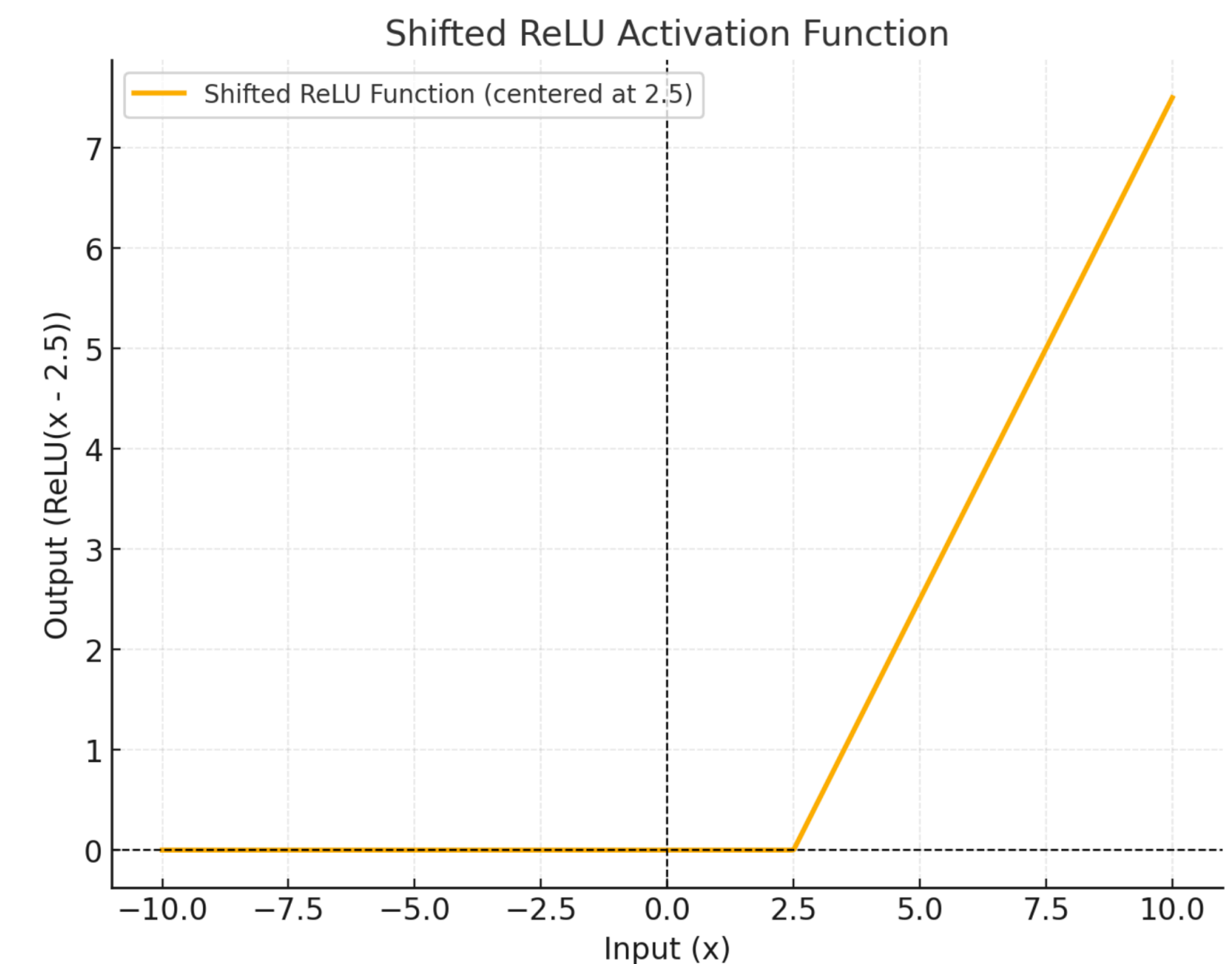
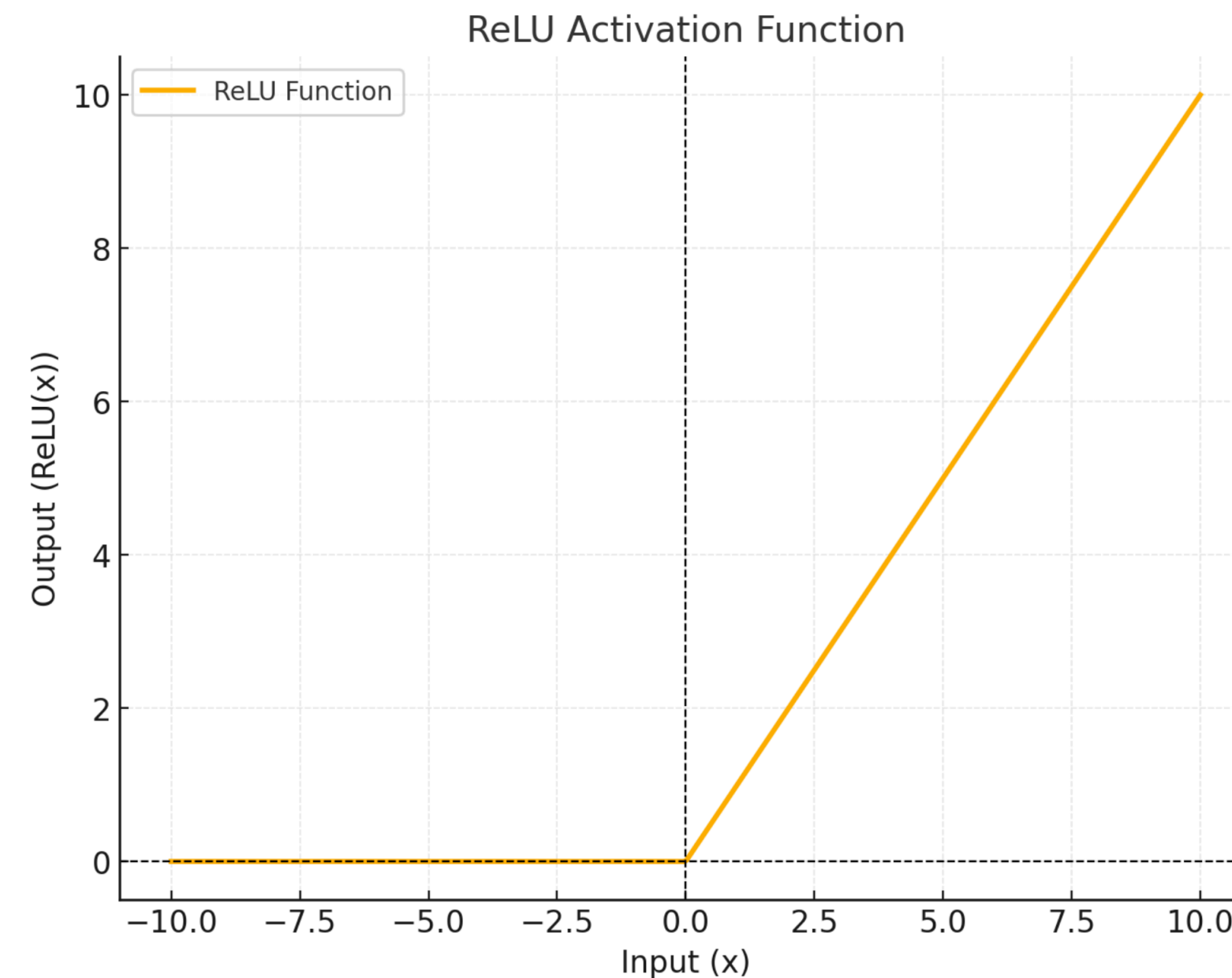
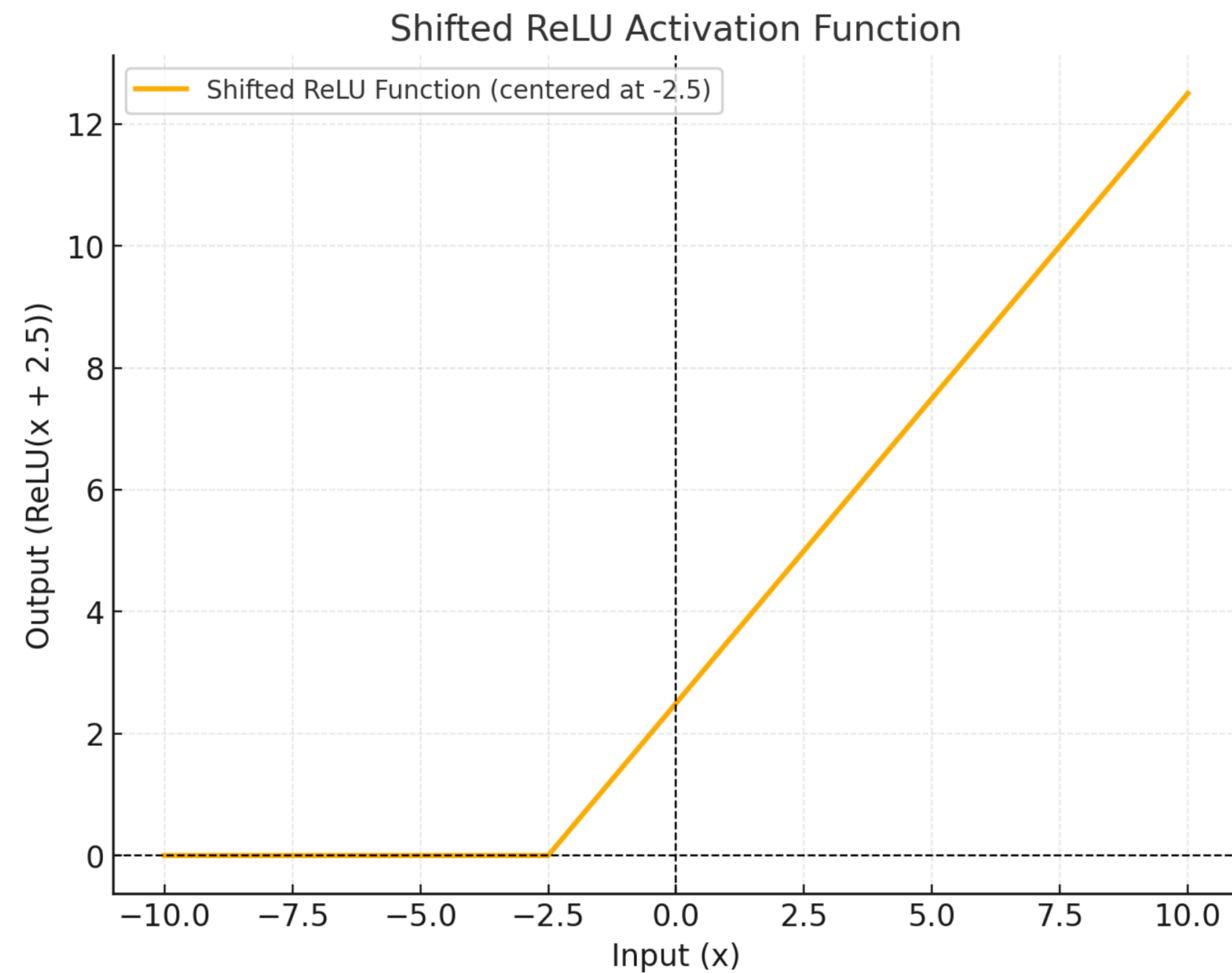
- În rețelele neuronale, intrările negative pot fi interpretate ca semnale mai puțin relevante pentru activarea unui neuron
- ReLU filtrează aceste semnale, păstrând doar valorile pozitive, considerate mai importante pentru propagarea informației în rețea

Funcția de activare



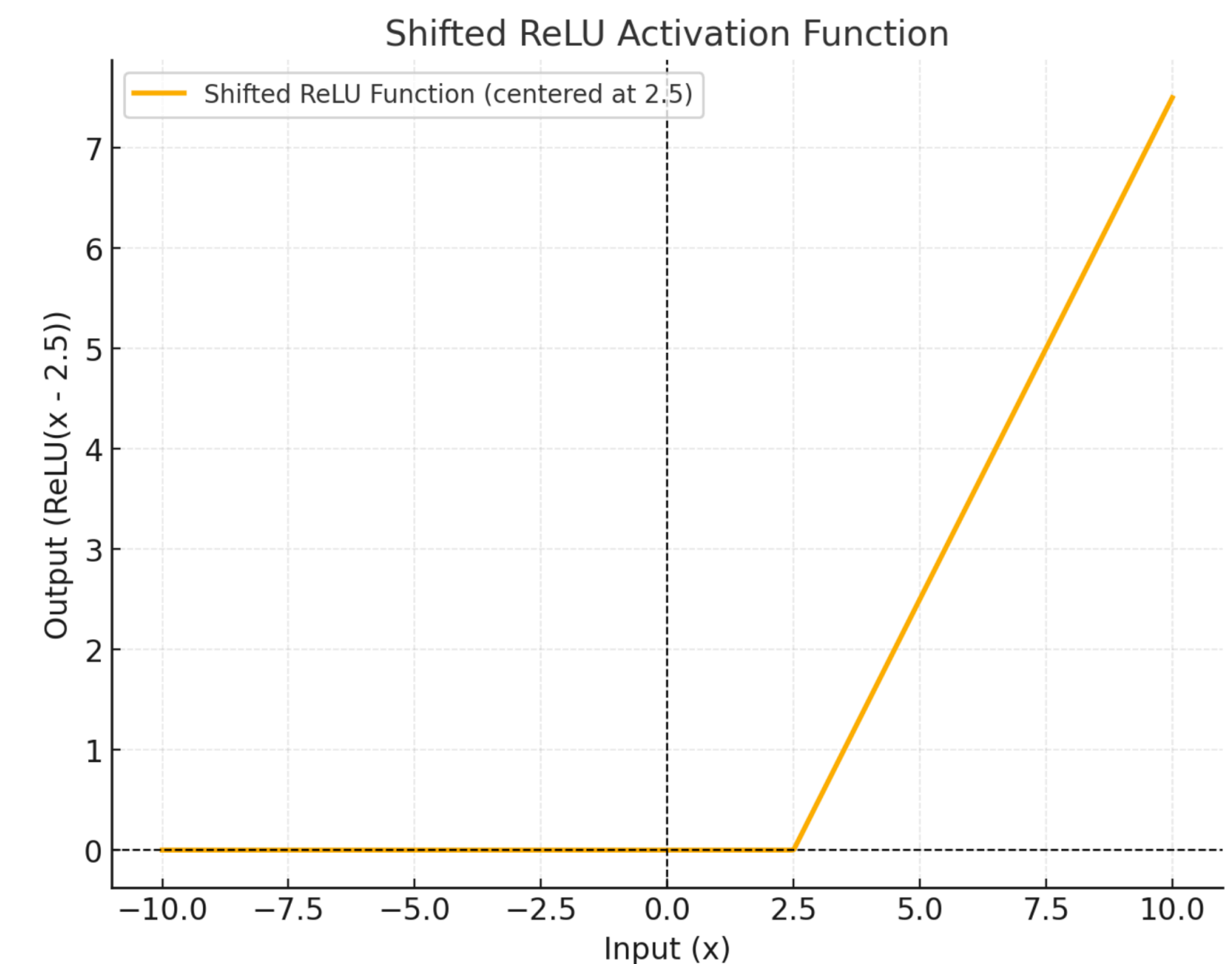
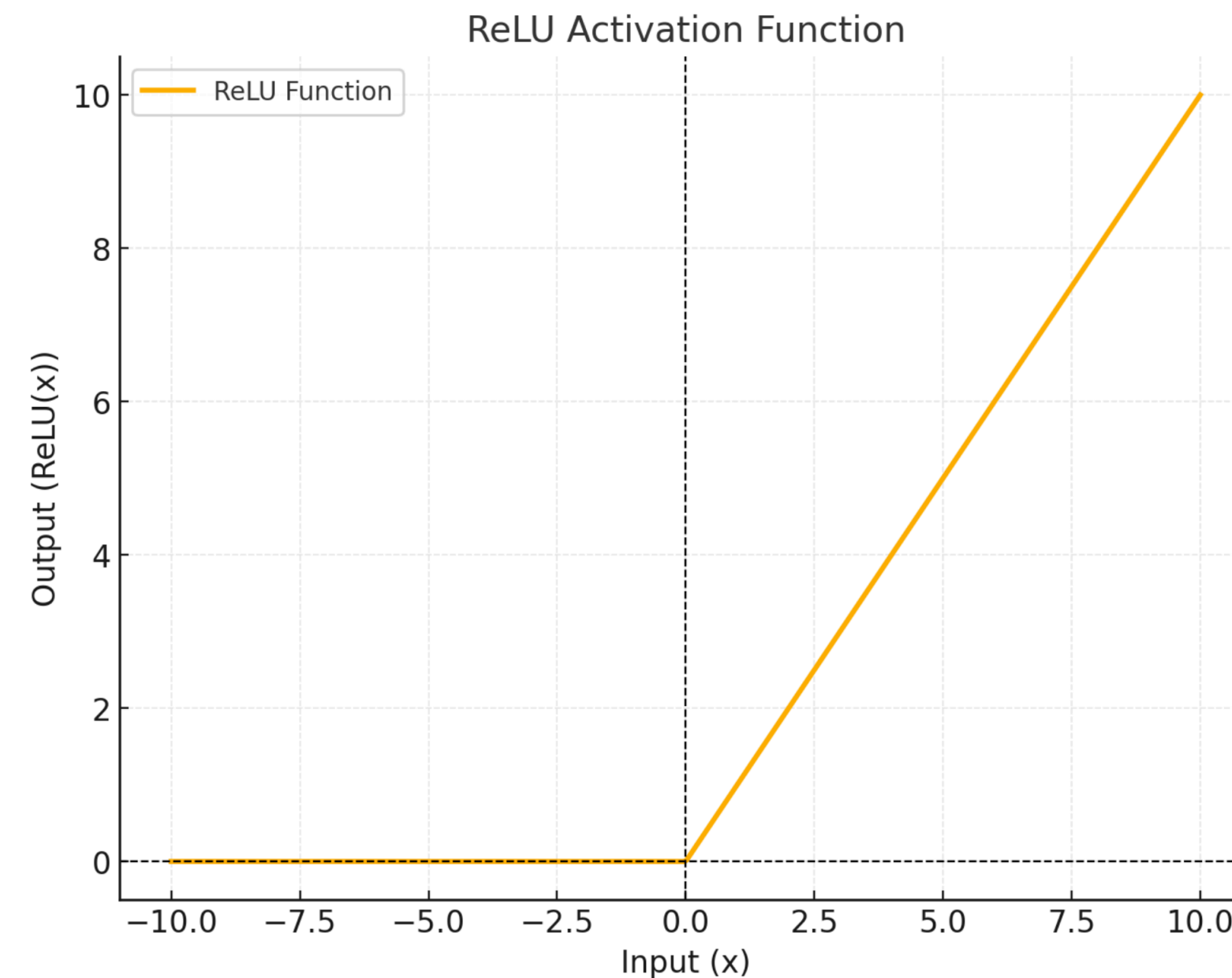
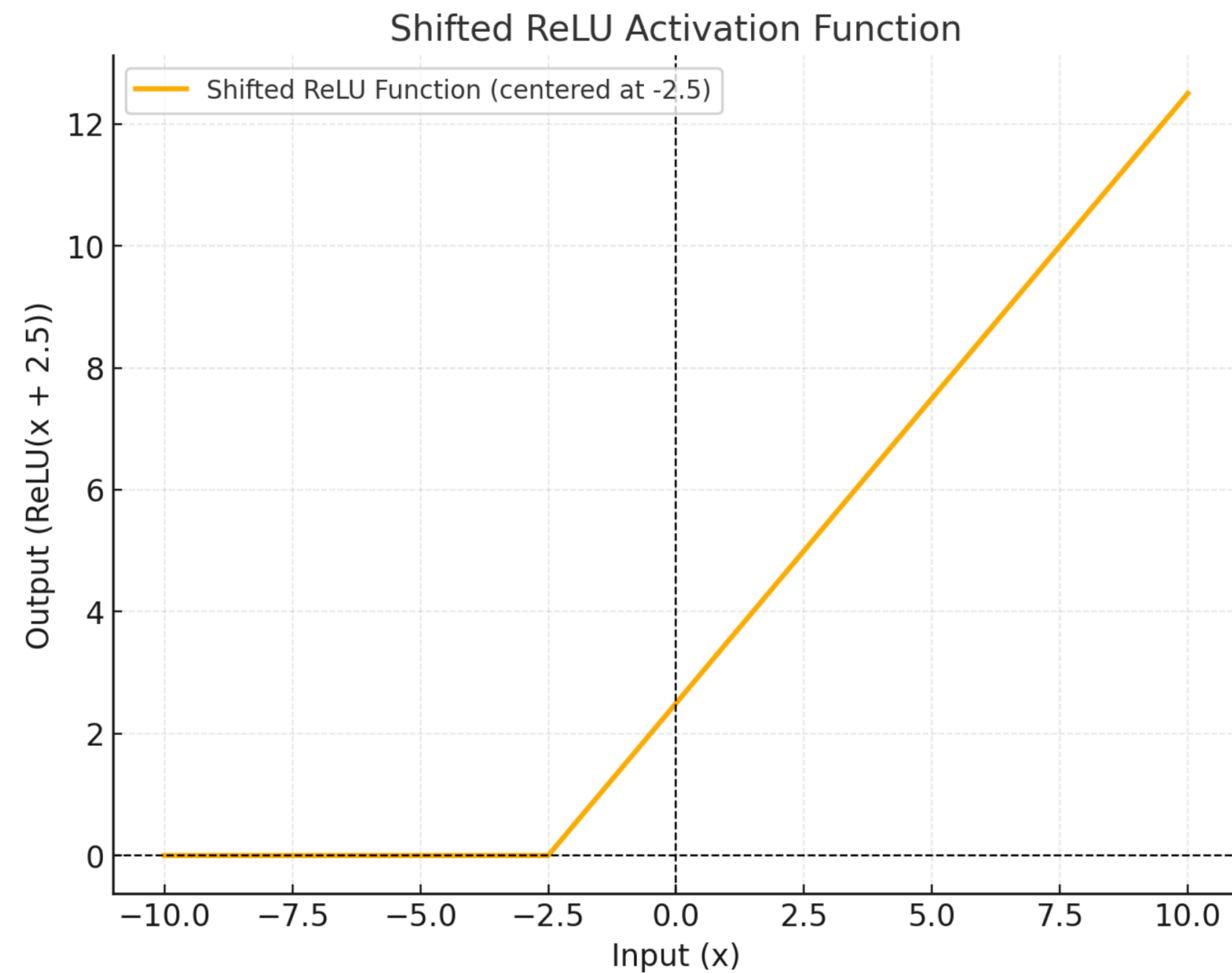
- Combinația dintre partea constantă ($x < 0, y = 0$) și partea liniară ($x \geq 0, y = x$) face ca ReLU să fie o funcție non-liniară deoarece nu este liniară pe întregul domeniu
- Astfel, reusim să modelăm funcții mai complexe

Rolul bias-ului în funcția de activare



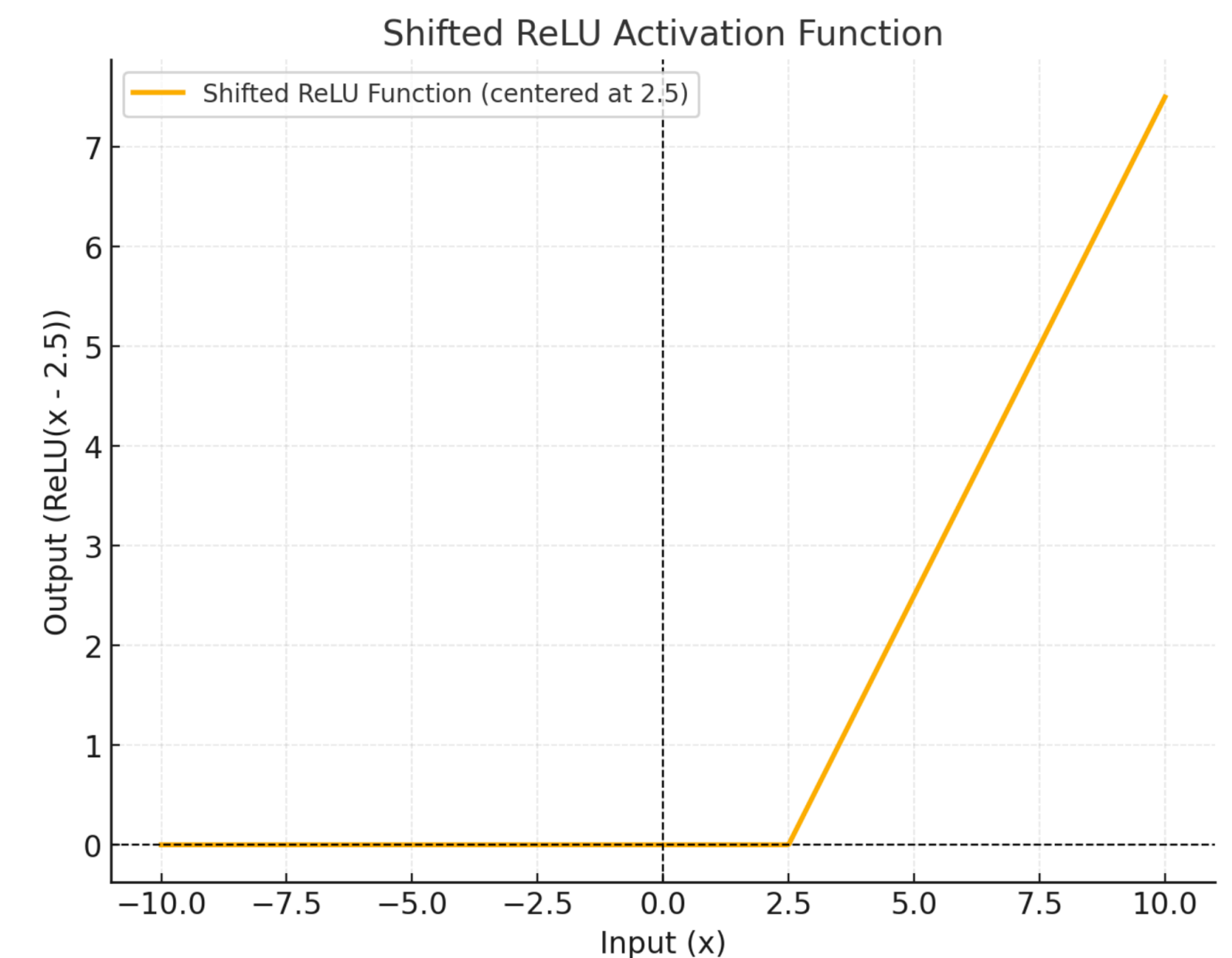
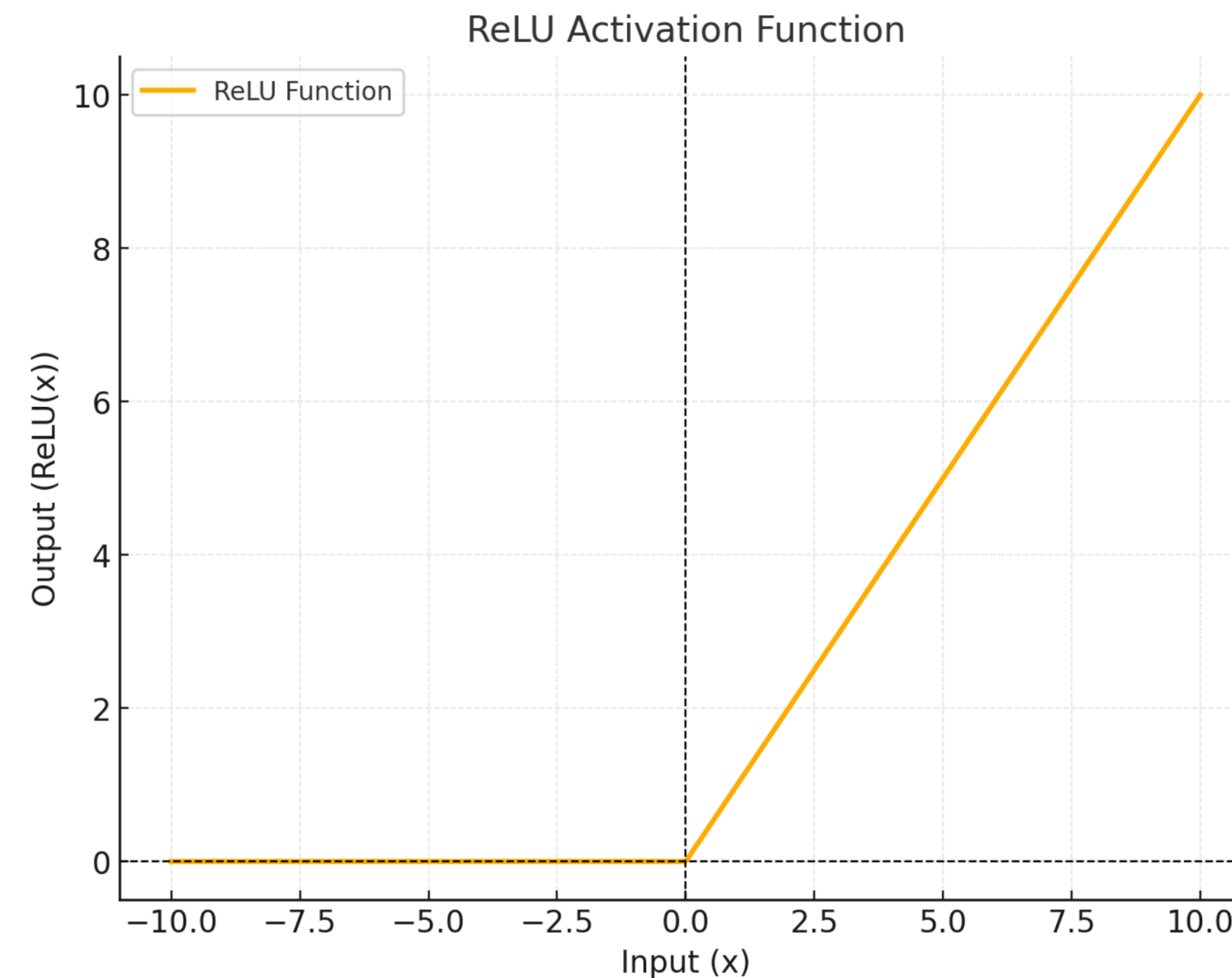
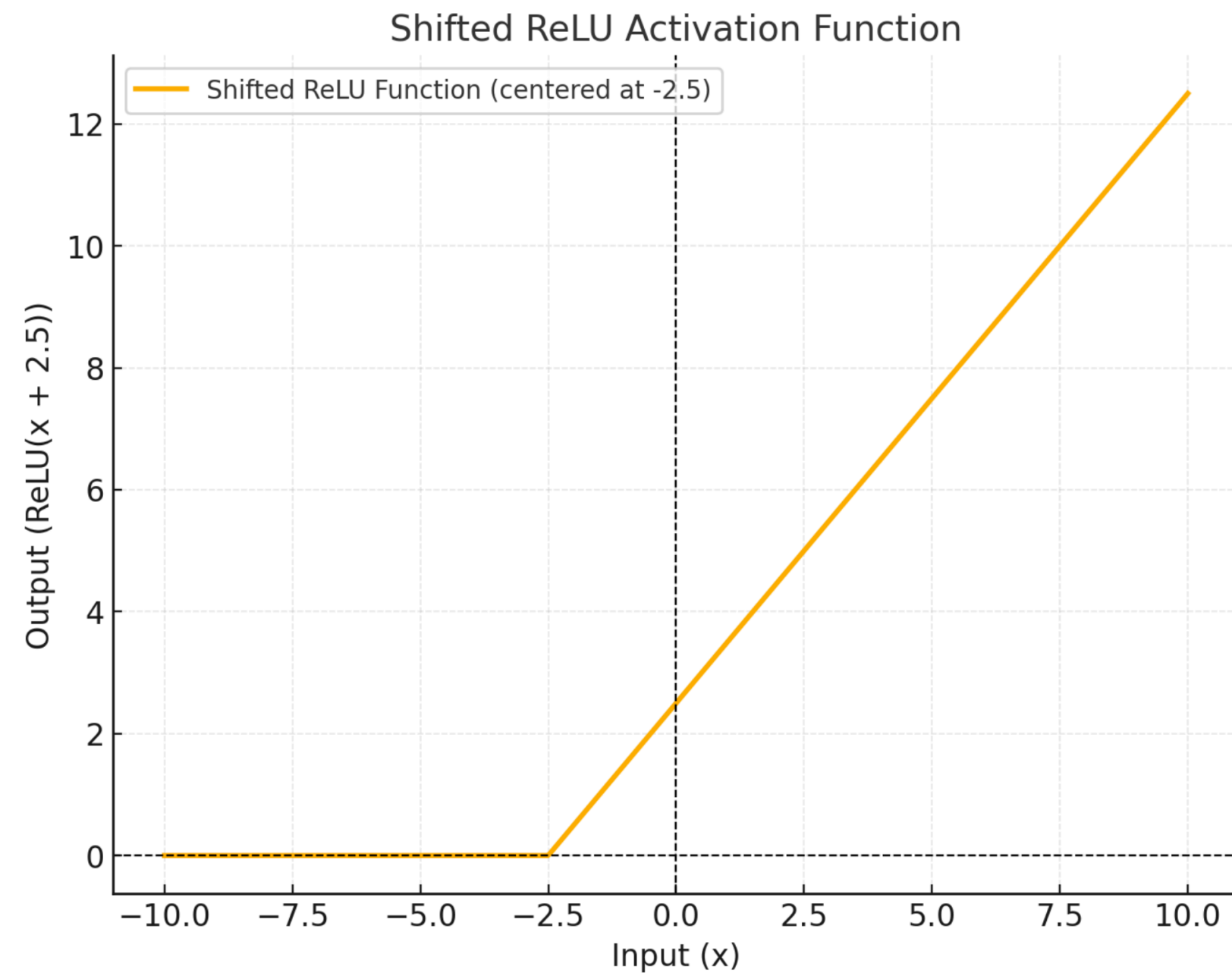
- Bias-ul poate fi comparat cu pragul de activare al neuronului biologic
- Este acel nivel minim de stimulare necesar pentru ca un neuron să se activeze și să trimită un semnal mai departe

Rolul bias-ului în funcția de activare



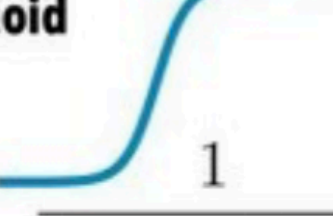
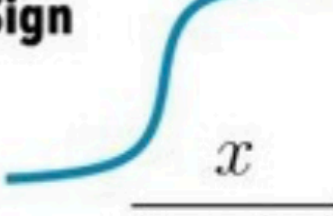
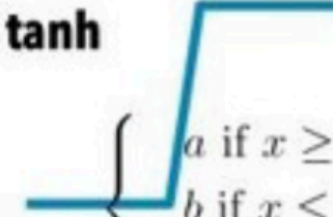
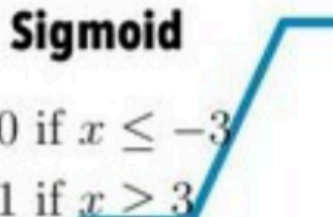
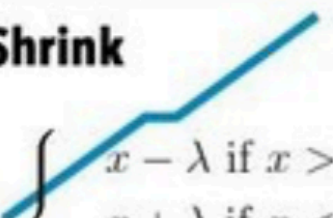
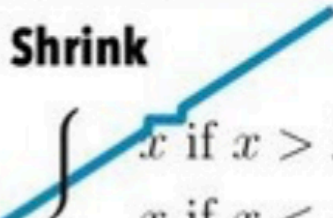
- Practic, bias-ul este “offset-ul” care determină cât de ușor sau greu se activează un neuron
- Fără bias, pragul implicit este 0, deoarece intrările ponderate sunt pur și simplu comparate cu 0 de către funcția de activare

Rolul bias-ului în funcția de activare



- Deci, fără bias, rețeaua pierde flexibilitatea de a adapta pragul în funcție de date sau de context
- De aceea, bias-ul este foarte important și își are rolul esențial de adaptabilitate a pragului din funcția de activare

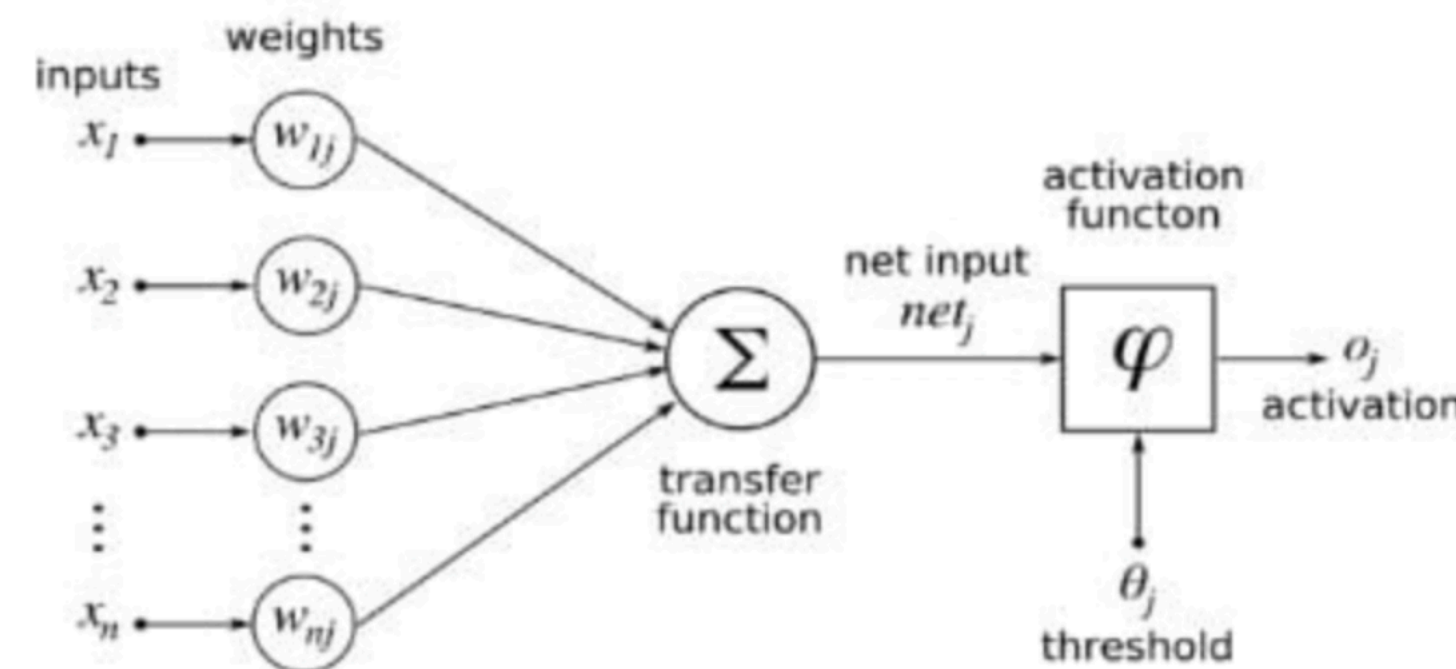
Alte exemple de funcții de activare

ReLU  $\max(0, x)$	GELU  $\frac{x}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + ax^3) \right) \right)$	PReLU  $\max(0, x)$
ELU  $\begin{cases} x & \text{if } x > 0 \\ \alpha(x \exp x - 1) & \text{if } x < 0 \end{cases}$	Swish  $\frac{x}{1 + \exp -x}$	SELU  $\alpha(\max(0, x) + \min(0, \beta(\exp x - 1)))$
SoftPlus  $\frac{1}{\beta} \log(1 + \exp(\beta x))$	Mish  $x \tanh \left(\frac{1}{\beta} \log(1 + \exp(\beta x)) \right)$	RReLU  $\begin{cases} x & \text{if } x \geq 0 \\ ax & \text{if } x < 0 \text{ with } a \sim \mathcal{R}(l, u) \end{cases}$
HardSwish  $\begin{cases} 0 & \text{if } x \leq -3 \\ x & \text{if } x \geq 3 \\ x(x+3)/6 & \text{otherwise} \end{cases}$	Sigmoid  $\frac{1}{1 + \exp(-x)}$	SoftSign  $\frac{x}{1 + x }$
Tanh  $\tanh(x)$	Hard tanh  $\begin{cases} a & \text{if } x \geq a \\ b & \text{if } x \leq b \\ x & \text{otherwise} \end{cases}$	Hard Sigmoid  $\begin{cases} 0 & \text{if } x \leq -3 \\ 1 & \text{if } x \geq 3 \\ x/6 + 1/2 & \text{otherwise} \end{cases}$
Tanh Shrink  $x - \tanh(x)$	Soft Shrink  $\begin{cases} x - \lambda & \text{if } x > \lambda \\ x + \lambda & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$	Hard Shrink  $\begin{cases} x & \text{if } x > \lambda \\ x & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$

Reprezentarea unui neuron artificial

- Regresia liniară reprezintă baza unui neuron artificial
- Regresia liniară ia următoarea formă generală:

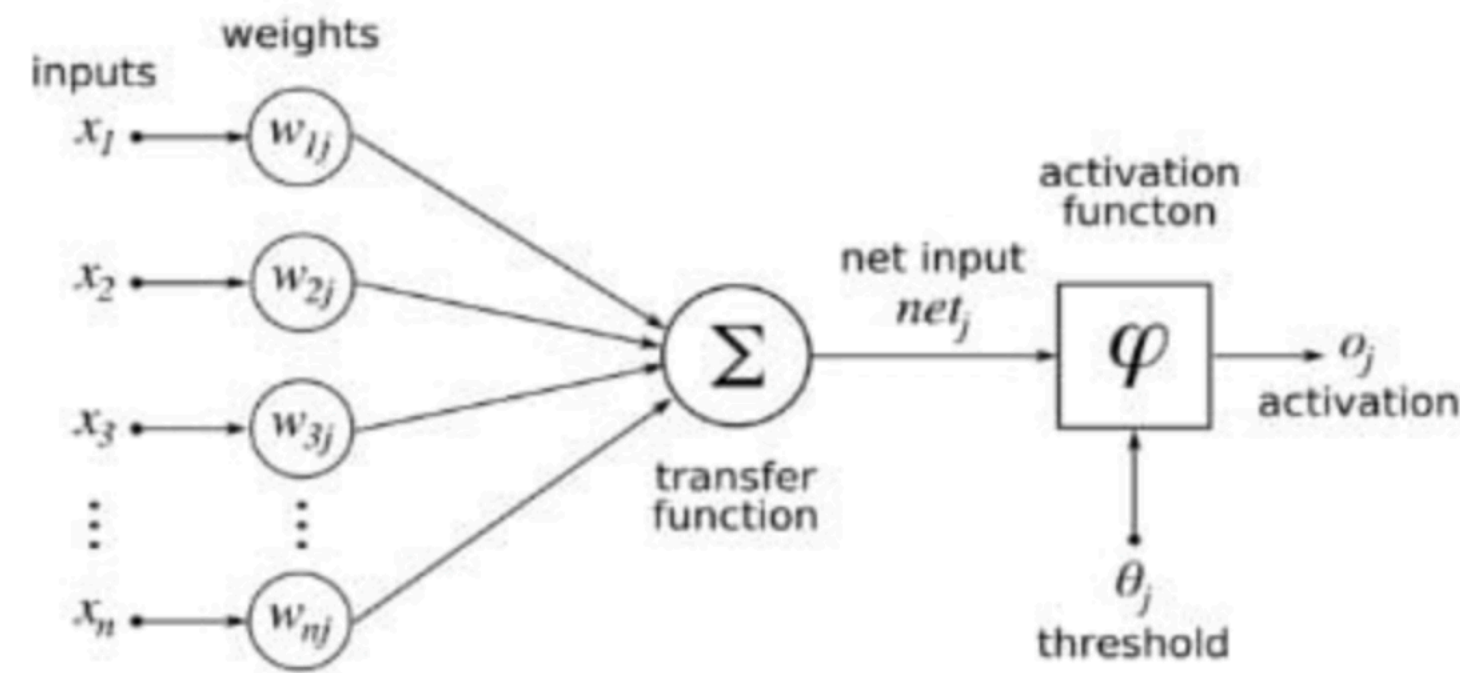
$$y = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n$$



- Tot ce lipsește acestei reprezentări este o funcție de activare

Reprezentarea unui neuron artificial

$$y = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n$$



- Tot ce lipsește acestei reprezentări este o funcție de activare
- Prezența funcției de activare transformă regresia liniară într-un neuron artificial

$$y = \sigma(w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n)$$

- Aici, σ reprezintă o funcție de activare (poate fi ReLU, poate fi oricare alta de pe slide-urile trecute)

Reprezentarea unui neuron artificial

$$y = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n$$

- În PyTorch:

```
[ ] # `nn` is an abbreviation for neural networks
    from torch import nn

    net = nn.Sequential(nn.Linear(2, 1))
```

- `nn.Linear(n,m)` reprezintă un neuron artificial cu 2 intrări și o ieșire
- `nn.Sequential()` este folosit pentru a lega mai multe operații unul după altul

Reprezentarea unui neuron artificial

- `nn.Sequential()` este folosit pentru a lega mai multe operații unul după altul
- Spre exemplu, `nn.Linear()` primește ca intrare un tensor de dimensiunea 1 (vector) ca intrare
- O operație utilă pe care o realizăm adesea e cea de flatten, unde luăm un tablou multidimensional / tensor de n-dimensiuni și îl transformăm într-un tensor de dimensiunea 1

```
[ ] net = nn.Sequential(nn.Flatten(), nn.Linear(784, 1))
```

- În acest caz, `nn.Flatten` se ocupă fix de operația descrisă mai sus

Învățarea se face cu un cost

- Pentru ca un neuron artificial să învețe, este nevoie de o funcție de cost
- Funcția de cost ajută la reglarea ponderilor neuronului astfel încât să se potrivească mai bine cu datele de antrenare

```
[ ] loss = nn.MSELoss()
```

- Pentru regresia liniară, o funcție de cost folosită des este eroarea medie pătratică

Învățarea se face cu un cost

Suprafața în metrii pătrați

prețul unui apartament

50

35000

60

40000

70

48000

80

55000

90

62000

100

70000

110

75000

120

85000

Învățarea se face cu un cost

Suprafața în metrii pătrați	prețul unui apartament	prețul unui apartament (prezis)
50	35000	36850
60	40000	40050
70	48000	49128
80	55000	54998
90	62000	61999
100	70000	69988
110	75000	77000
120	85000	85002

Eroarea medie pătratică

prețul unui apartament		prețul unui apartament (prezis)
$\frac{1}{8} [(35000$	-	$36850)^2 +$
$(40000$	-	$40050)^2 +$
$(48000$	-	$49128)^2 +$
$(55000$	-	$54998)^2 +$
$(62000$	-	$61999)^2 +$
$(70000$	-	$69988)^2 +$
$(75000$	-	$77000)^2 +$
$(85000$	-	$85002)^2] = \text{cost}$

Neuroni artificiali pentru clasificare

- Pentru clasificare, se folosesc adesea funcțiile de activare sigmoid și softmax pentru transformarea ieșirii regresiei liniare în probabilități
- În cazul funcției de activare sigmoid, regresia liniară se transformă în regresia logistică
- În cazul funcției de activare softmax, regresia liniară se transformă în regresia softmax
- În acest caz, pentru clasificare, folosim des funcția de cost cross-entropy:

```
net = nn.Sequential(nn.Linear(n, m))  
loss = nn.CrossEntropyLoss()
```

Neuroni artificiali pentru clasificare

```
net = nn.Sequential(nn.Linear(n, m))  
loss = nn.CrossEntropyLoss()
```

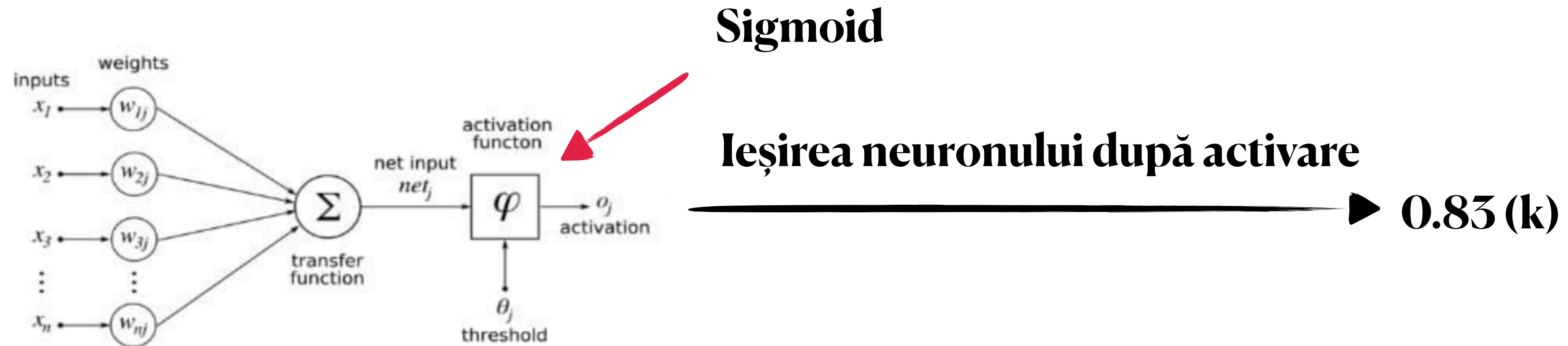
- Mai sus, dacă $m = 1$ avem regresia logistică și dacă $m > 1$ avem regresia softmax
- În regresia logistică, modelul realizează clasificare binară și dă ca ieșire o singură valoare. O valoare mai mare înseamnă că clasa „True” are șanse mai mari să se întâmple, iar o valoare mai mică înseamnă că clasa „False” are șanse mai mari să se întâmple
- În regresia softmax, modelul dă ca ieșire M probabilități, care însumate dau valoarea 1. E practic o distribuție de probabilități care spune care clasă e mai predominantă decât alta

Neuroni artificiali pentru clasificare

- În regresia logistică, modelul realizează clasificare binară și dă ca ieșire o singură valoare. O valoare mai mare înseamnă că clasa „True” are șanse mai mari să se întâmple, iar o valoare mai mică înseamnă că clasa „False” are șanse mai mari să se întâmple
- În regresia softmax, modelul dă ca ieșire M probabilități, care însumate dau valoarea 1. E practic o distribuție de probabilități care spune care clasă e mai predominantă decât alta
- Cross-entropy măsoară cât de bine aceste probabilități reflectă etichetele reale

Neuroni artificiali pentru clasificare

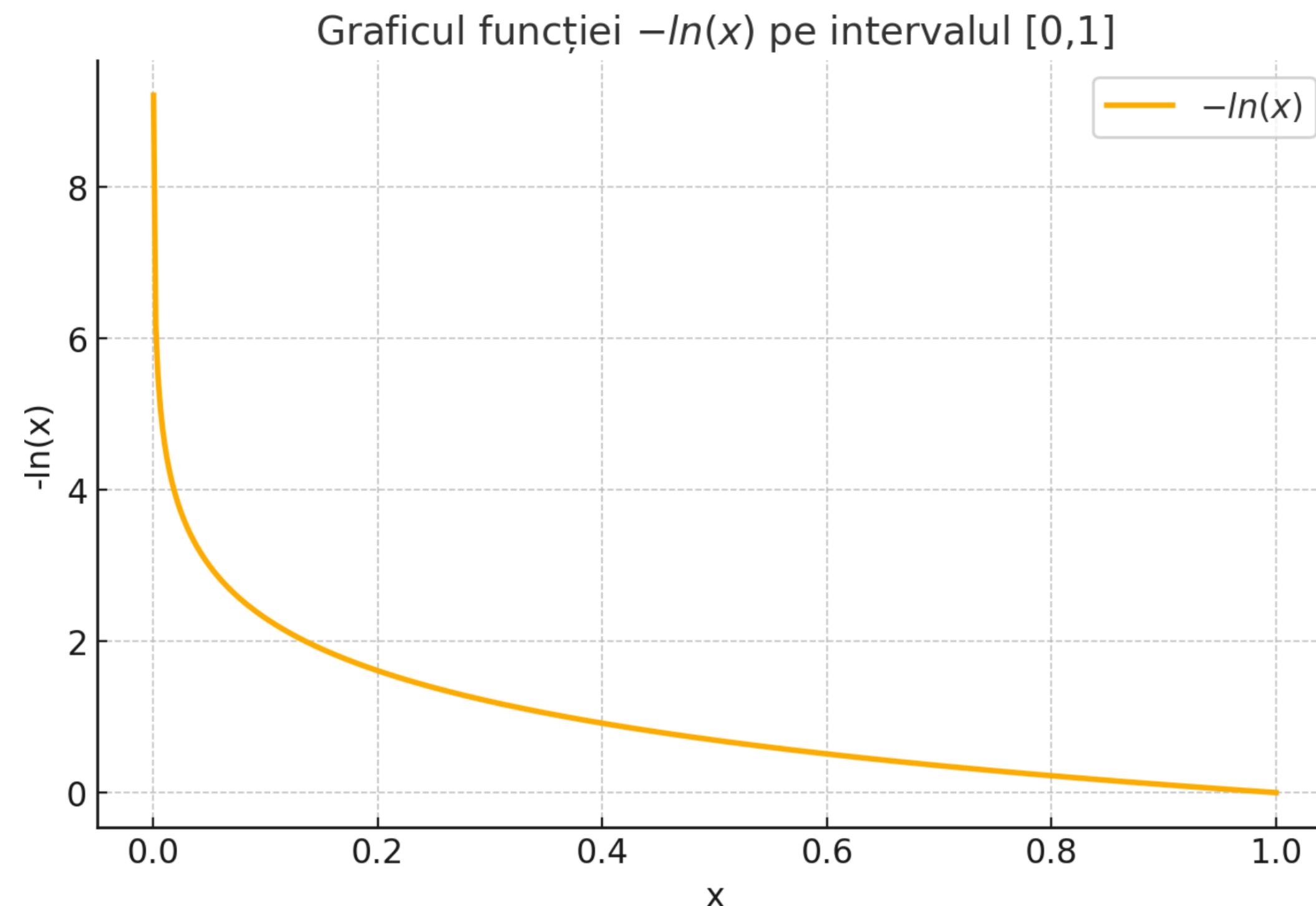
- Pentru regresia logistică:



- Dacă y (ground truth) = 1, atunci se încearcă minimizarea funcției $-\log(k)$
- Motivul este acela ca minimizarea $-\log(k)$ implică aducerea k spre 1, deoarece atunci $\log(k)$ devine $\log(1)=0$, si funcția de cost este perfect minimizată

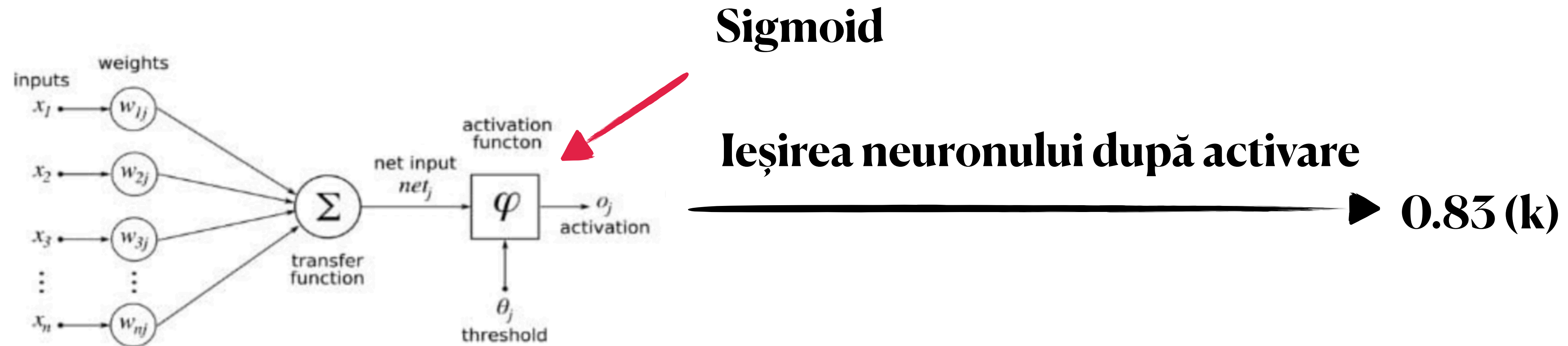
Neuroni artificiali pentru clasificare

- Dacă y (ground truth) = 1, atunci se încearcă minimizarea funcției $-\log(k)$
- Motivul este acela ca minimizarea $-\log(k)$ implică aducerea k spre 1, deoarece atunci $\log(k)$ devine $\log(1)=0$, si funcția de cost este perfect minimizată



Neuroni artificiali pentru clasificare

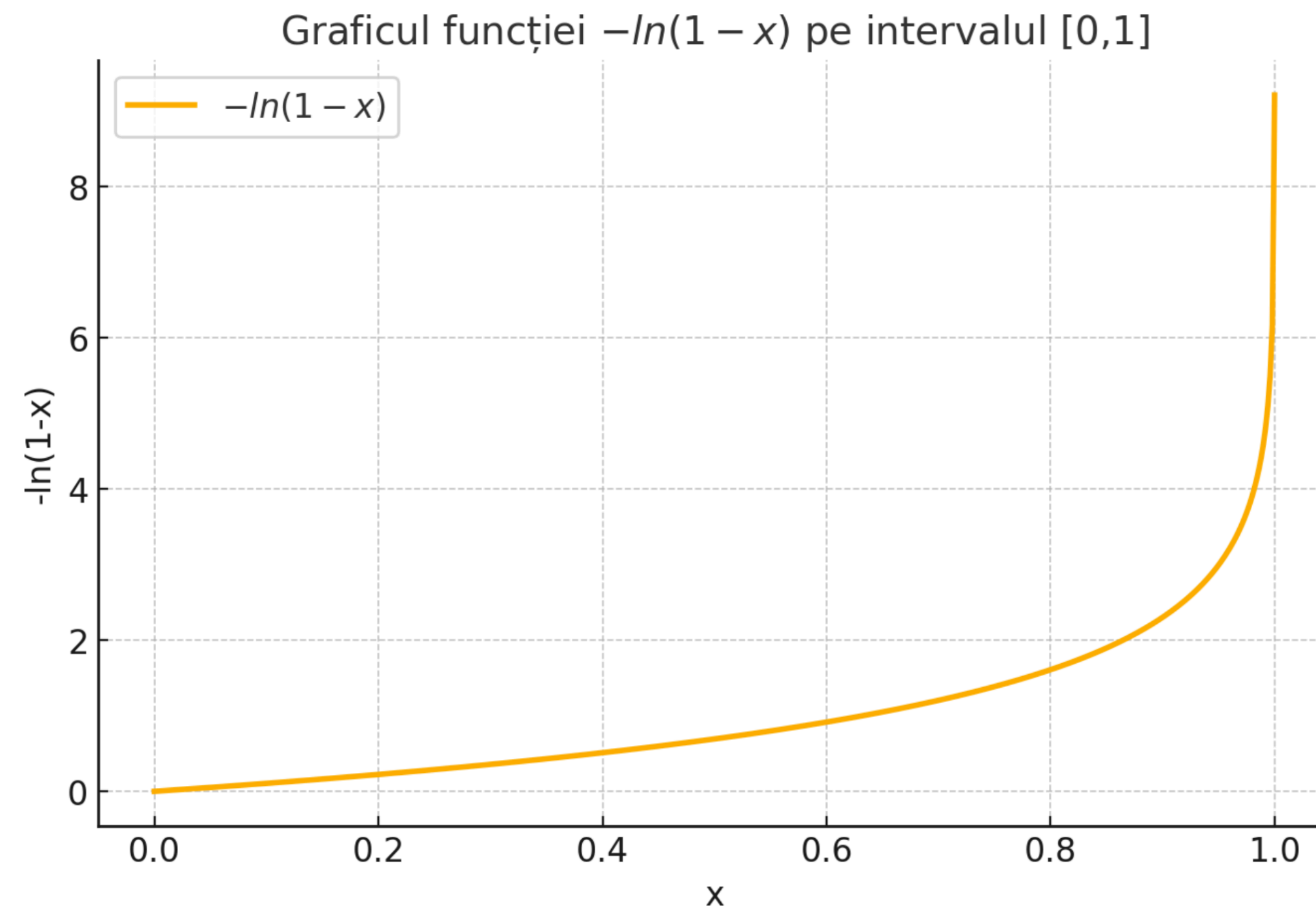
- Pentru regresia logistică:



- Dacă y (ground truth) = 0, atunci se încearcă minimizarea funcției $-\log(1-k)$
- Motivul este acela ca minimizarea $\log(1-k)$ implică aducerea k spre 0, deoarece atunci $-\log(1-k)$ devine $-\log(1)=0$, și funcția de cost este perfect minimizată

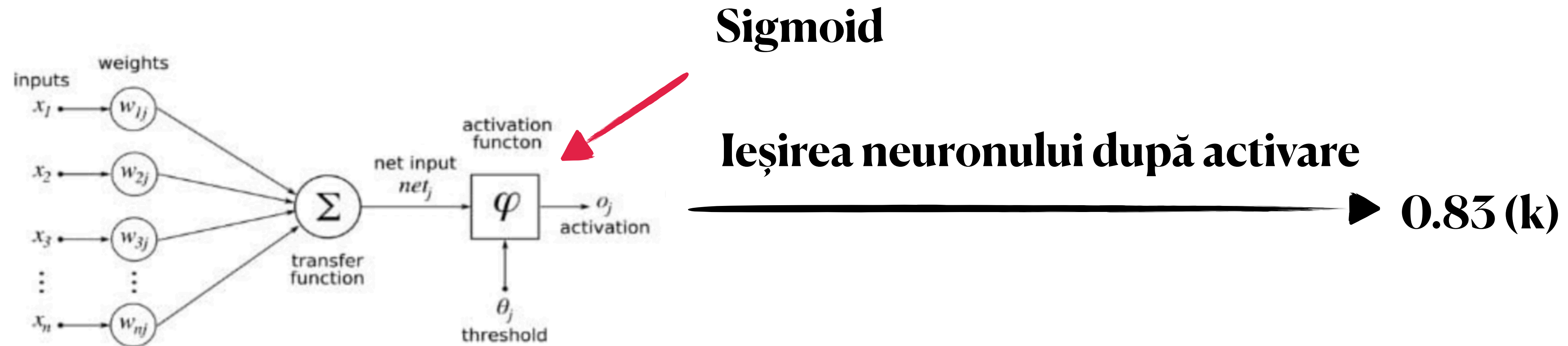
Neuroni artificiali pentru clasificare

- Dacă y (ground truth) = 0, atunci se încearcă minimizarea funcției $-\log(1-k)$
- Motivul este acela ca minimizarea $\log(1-k)$ implică aducerea k spre 0, deoarece atunci $-\log(1-k)$ devine $-\log(1)=0$, si funcția de cost este perfect minimizată



Neuroni artificiali pentru clasificare

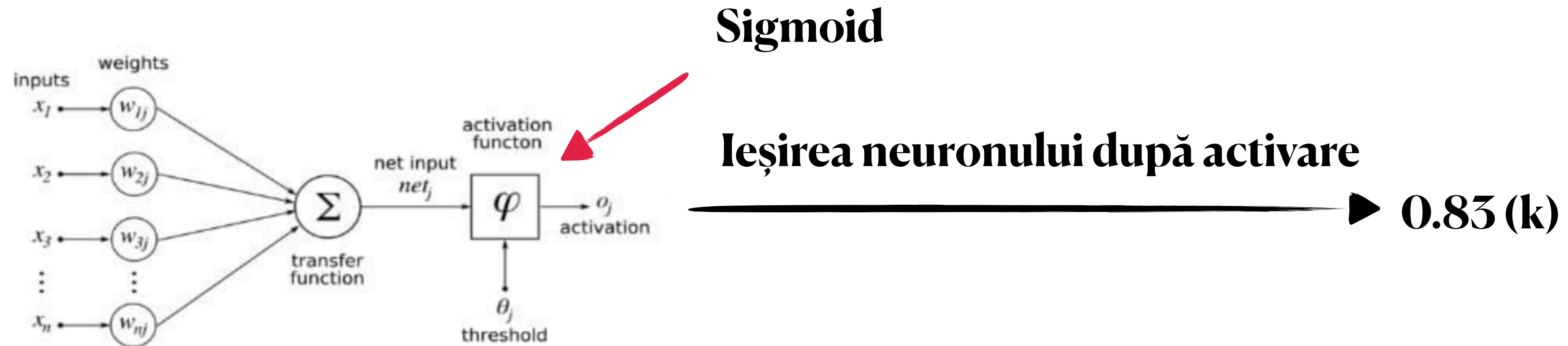
- Pentru regresia logistică:



- În cazul nostru, să zicem că modelul trebuia să prezică 0, adică $y = 0$, atunci costul curent este $-\log(1 - 0.83) = 0.77$, deci un cost mare, deoarece trebuia să prezică ceva mai aproape de 0
- Dacă modelul trebuia să prezică 1, adică $y = 1$, atunci costul curent este $-\log(0.83) = 0.08$, deci un cost mic și bun

Neuroni artificiali pentru clasificare

- Pentru regresia logistică:



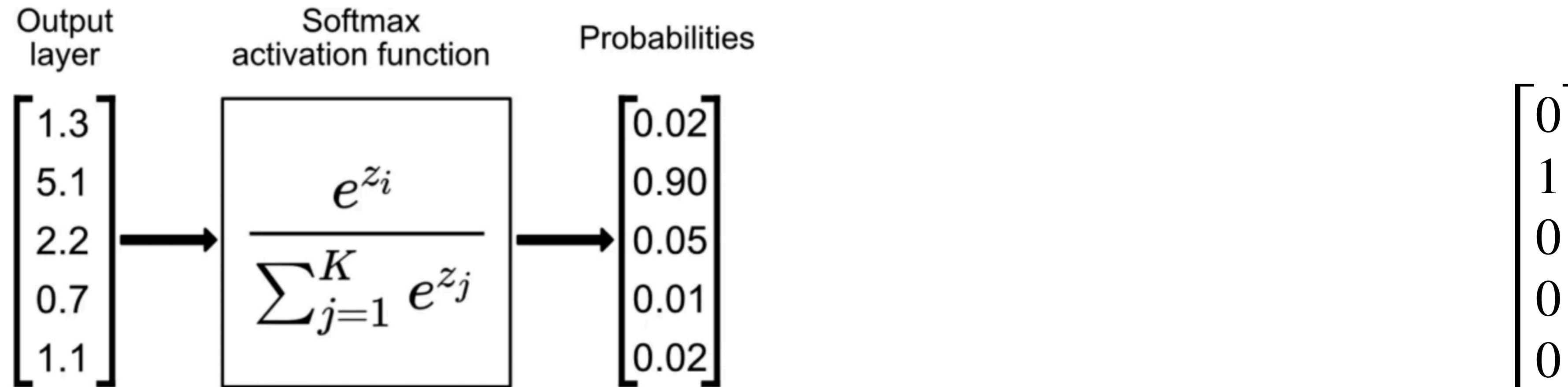
- Mai general, dacă avem un batch, formula pentru cross entropy în cazul regresiei logistice ia următoarea formă (m este numărul de exemplare dintr-un batch):

$$Eroare = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\text{sigmoid}(\sum (x^{(i)}))) + (1 - y^{(i)}) \log(1 - \text{sigmoid}(\sum (x^{(i)}))) \right]$$

Neuroni artificiali pentru clasificare

- Pentru regresia softmax pentru clasificarea multi-class:
ieșirea modelului după funcția de activare softmax

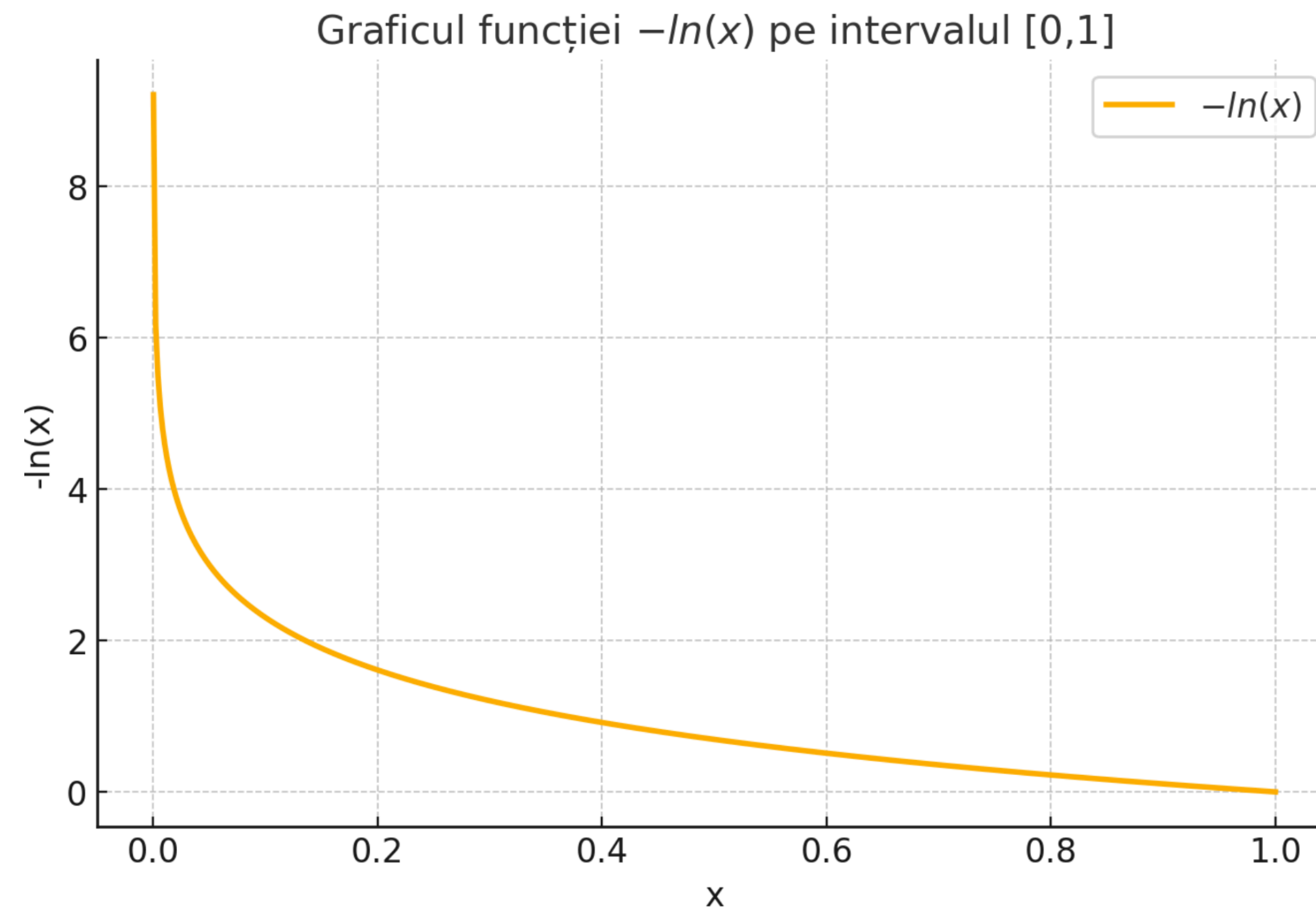
Adevărul de bază (ground truth)



- Se încearcă să se minimizeze funcția $-\log(x)$ în raport cu clasa considerată 1
- Mereu va exista o clasă 1 și restul claselor vor fi 0

Neuroni artificiali pentru clasificare

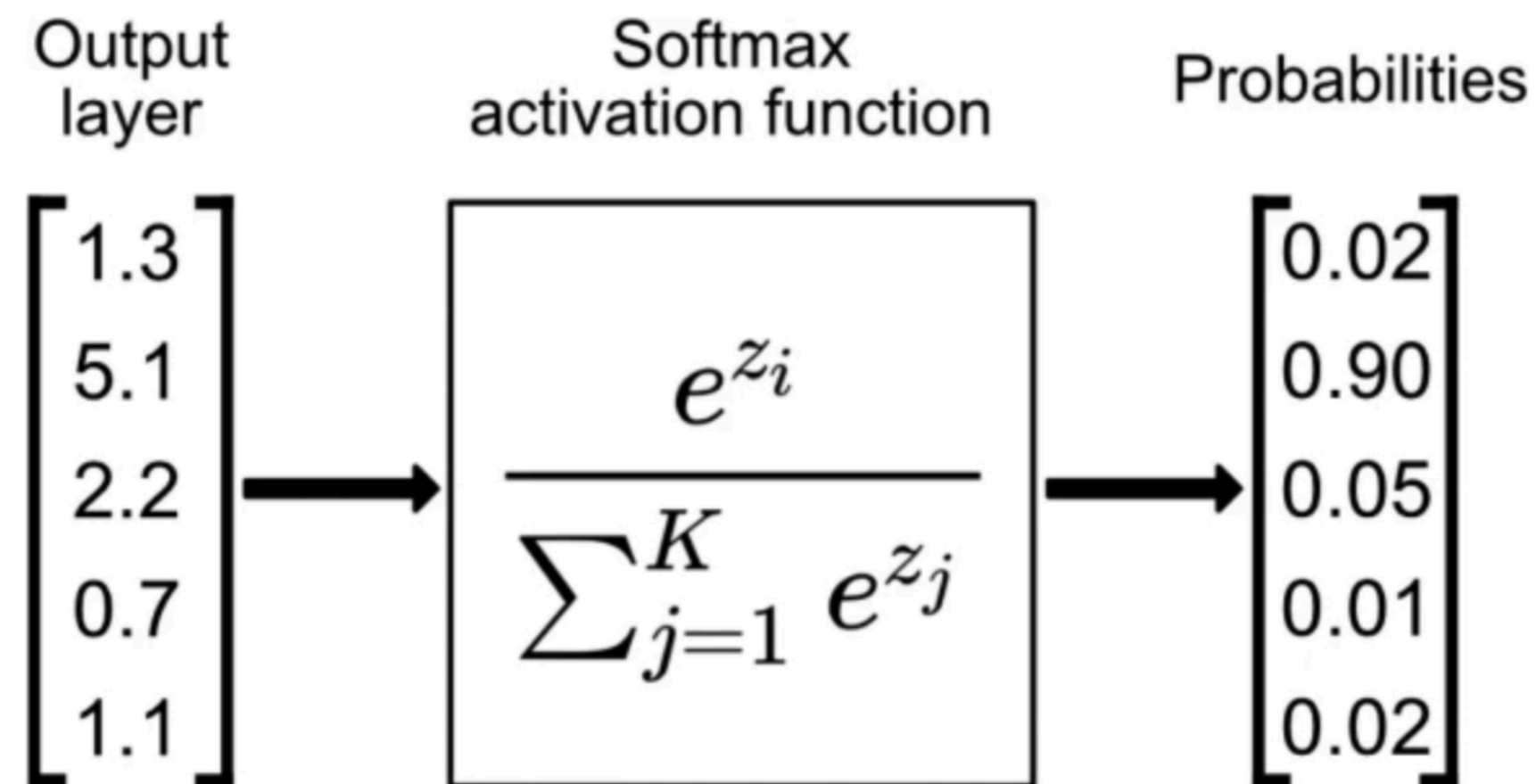
- Se încearcă să se minimizeze funcția $-\log(x)$ în raport cu toate probabilitățile



Neuroni artificiali pentru clasificare

- Pentru regresia softmax pentru clasificarea multi-class:

ieșirea modelului după funcția de activare softmax



Adevărul de bază (ground truth)

$$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

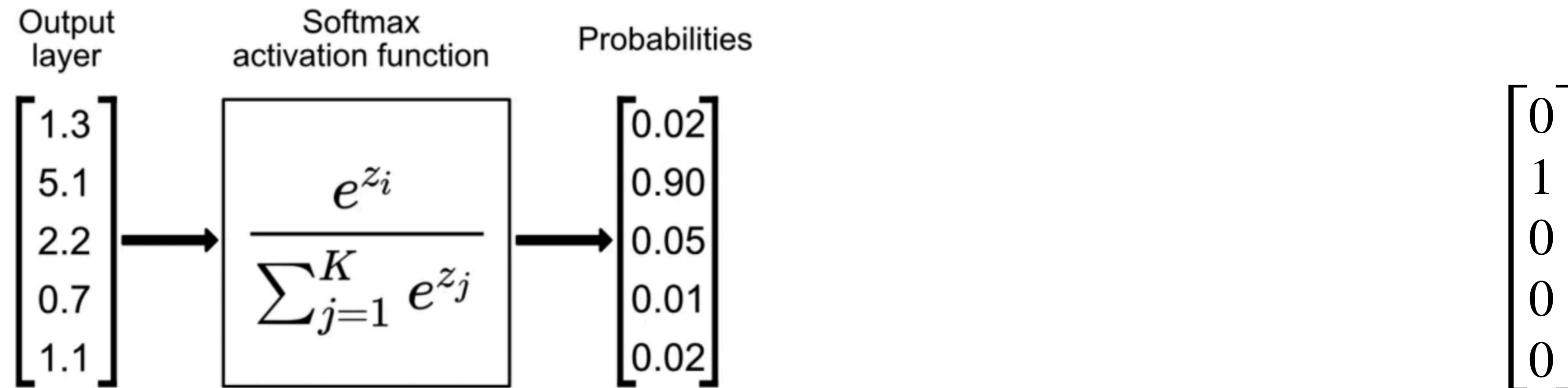
- În acest caz, costul va fi
$$-0 \cdot \log(0.02) + -1 \cdot \log(0.90) + -0 \cdot \log(0.05) + -0 \cdot \log(0.01) + -0 \cdot \log(0.02) = 0.1054$$
- Este mic în acest caz, deoarece clasa care e 1 este prezisă deja cu o probabilitate mare (0.90)

Neuroni artificiali pentru clasificare

- Pentru regresia softmax pentru clasificarea multi-class:

leșirea modelului după funcția de activare softmax

Adevărul de bază (ground truth)

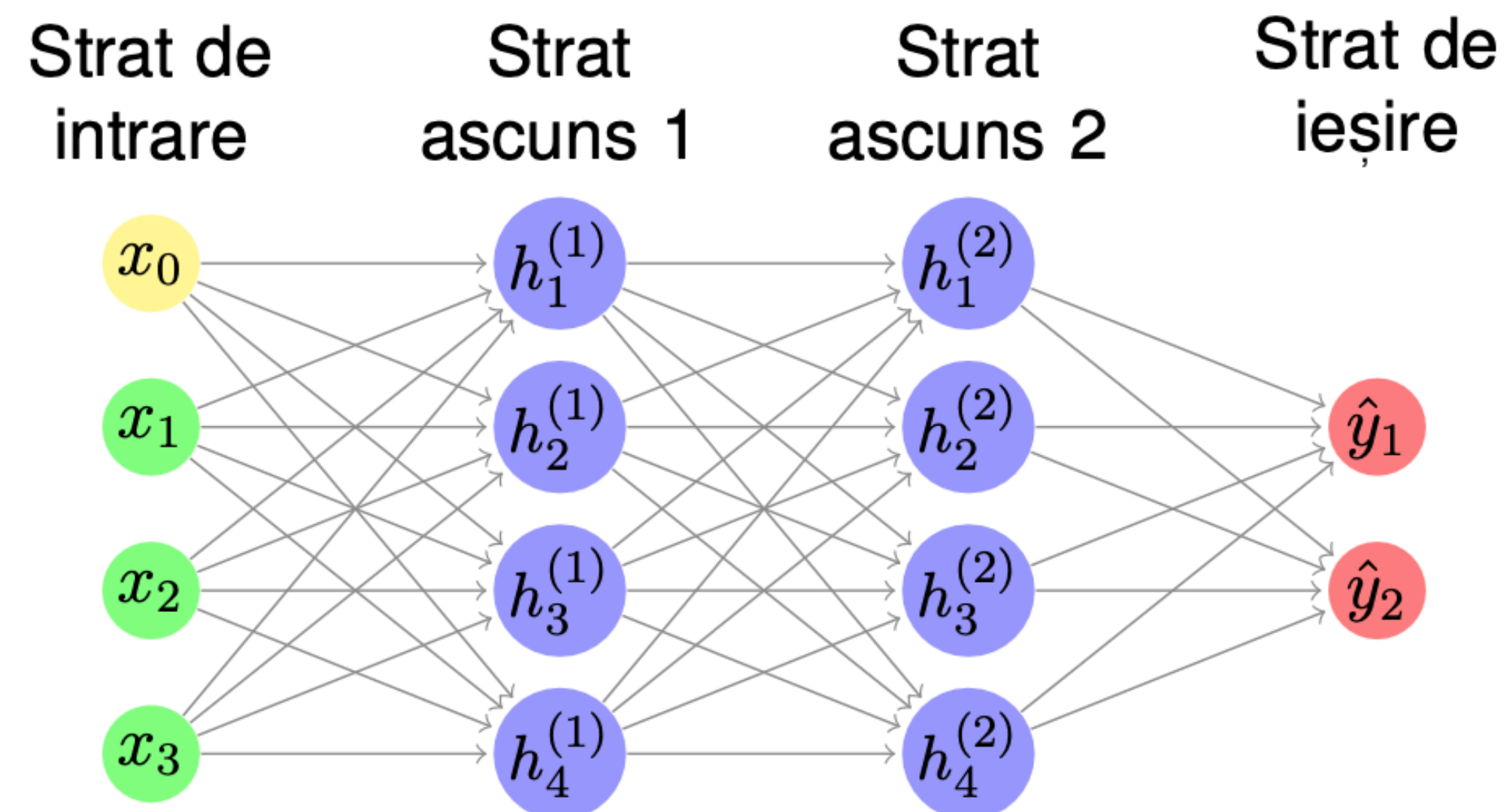


- Formula generală este (m reprezintă numărul de exemplare dintr-un batch, $y_c^{(i)}$ reprezintă adevărul de bază pentru a i-a prezicere și $p_c^{(i)}$ reprezintă prezicerea:

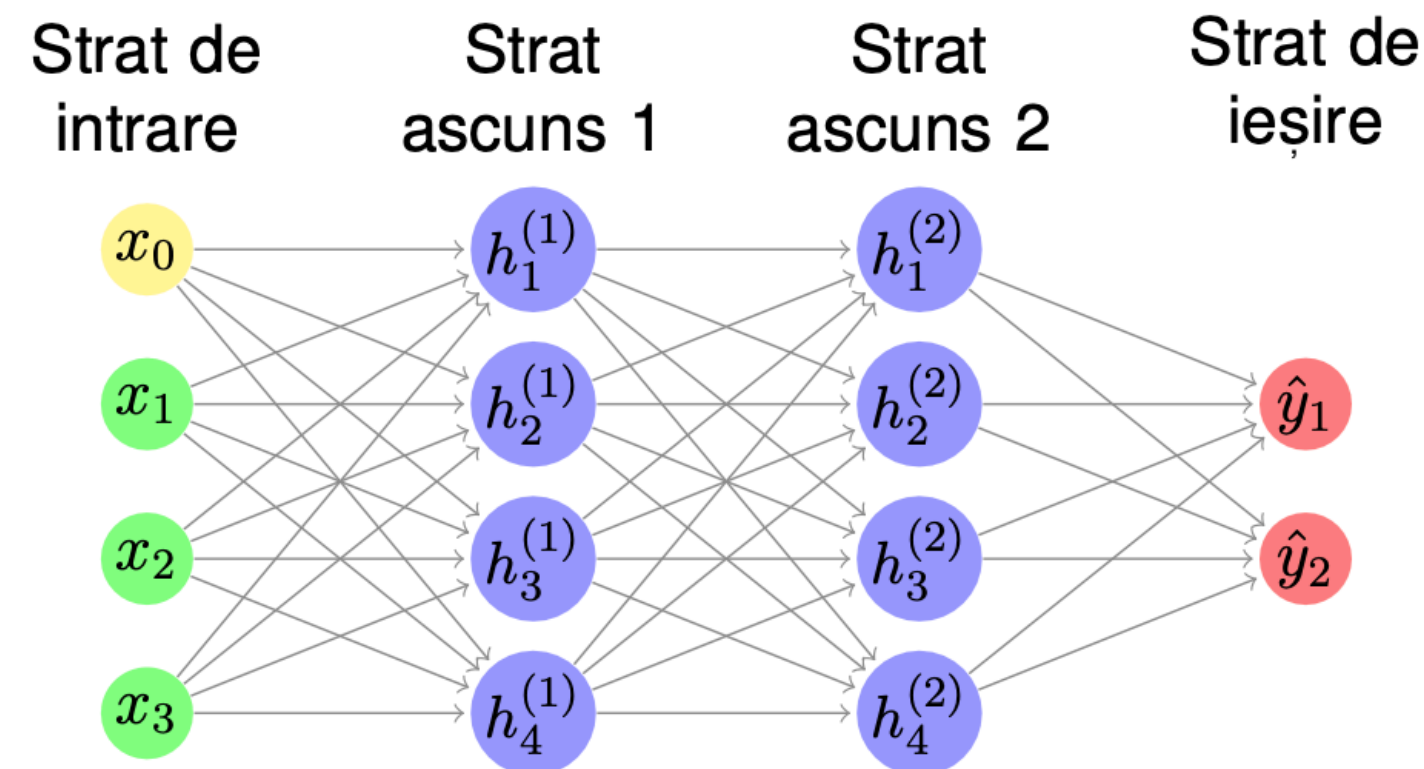
$$Eroare = -\frac{1}{m} \sum_{i=1}^m \sum_{c=1}^C y_c^{(i)} \log(p_c^{(i)})$$

Un singur neuron nu-i de ajuns

- Dacă problema noastră implică relații mai complexe între date, un singur neuron nu va fi suficient
- Astfel au fost introduși perceptronii multistrat (multilayer perceptrons):

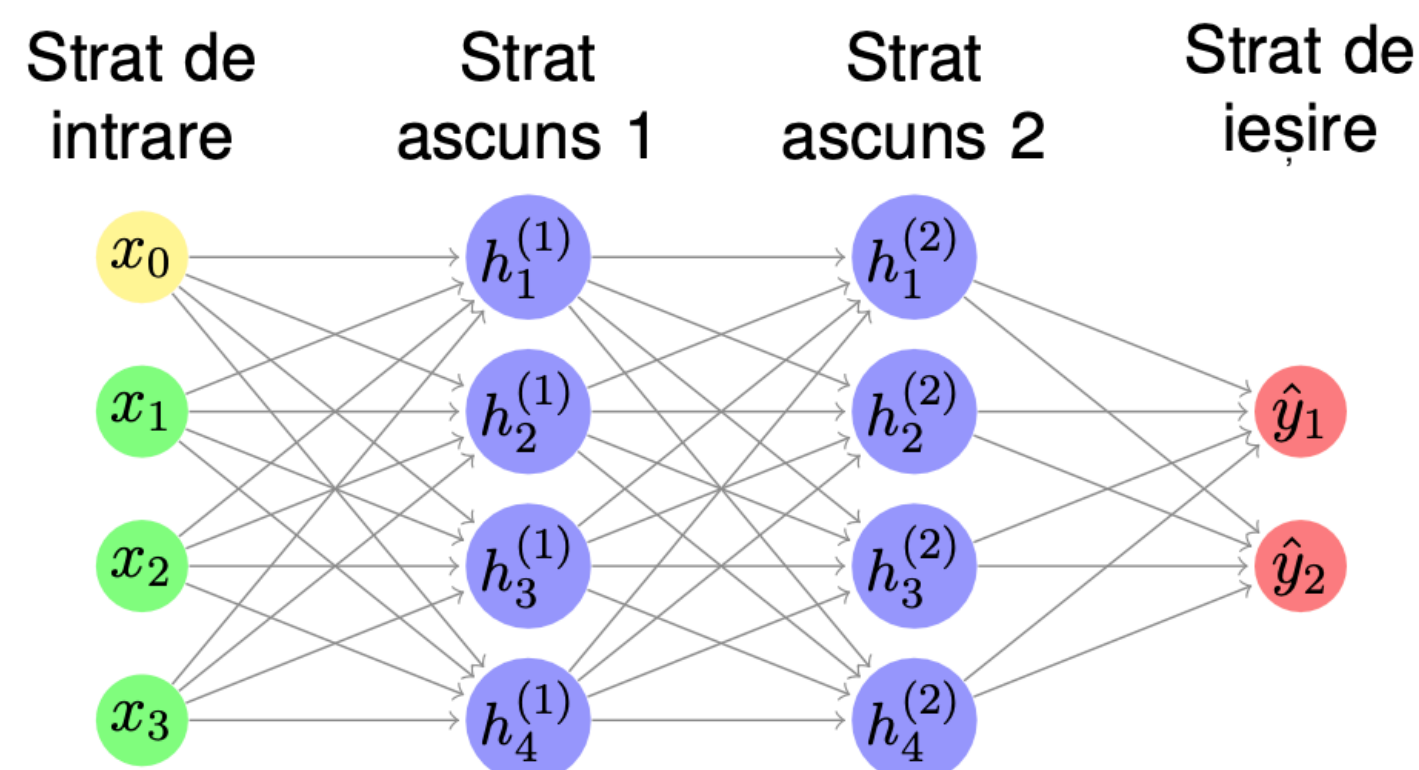


Un singur neuron nu-i de ajuns



- Un perceptron multistrat are:
 - Stratul de intrare: unde fiecare neuron primește datele brute
 - Straturile ascunse (hidden layers): unde datele sunt procesate și transformate printr-o combinație de ponderi, bias-uri și funcții de activare
 - Stratul de ieșire: unde rezultatul final este produs (de exemplu, valoarea numerică pentru regresie sau probabilități pentru clasificare)

Un singur neuron nu-i de ajuns



- Straturile ascunse permit modelului să construiască reprezentări complexe ale datelor
- Altfel spus, pornim de la $x_0, x_1, \dots, x_n$ care reprezintă caracteristicile exemplarului curent
- Ele sunt transformate în alte caracteristici în stratul ascuns 1, numite $h_1^{(1)}, h_2^{(1)}, \dots, h_k^{(1)}$
- Aceste noi caracteristici, încearcă să deducă lucruri din stratul precedent
- Ele vor fi transformate la rândul lor în alte caracteristici mai complexe în straturile ascunse ulterioare

Un singur neuron nu-i de ajuns

- De exemplu, recunoașterea emoțiilor din text în pozitiv și negativ
- Să zicem că avem textul „Sunt foarte fericit astăzi”
- Fiecare cuvânt este transformat într-un anumit vector (există metode)
- Primul strat ascuns învață caracteristici simple, cum ar fi detectarea cuvintelor cheie: „fericit” poate avea o greutate mare asociată cu un sentiment pozitiv, în timp ce „trist” ar avea o greutate pentru negativ
- Al doilea strat ascuns ar combina aceste caracteristici pentru a găsi tipare mai complexe: de exemplu, relația dintre „foarte” și „fericit” poate intensifica sentimentul pozitiv
- Al treilea strat ascuns învață context mai general, cum ar fi să ignore cuvintele care nu au legătură cu emoțiile („azi”, „sunt”) și să se concentreze pe părțile relevante ale textului
- Deci fiecare strat ascuns poate să se concentreze mai bine pe problema în cauză

Un singur neuron nu-i de ajuns

- În PyTorch, e foarte simplu să realizăm un perceptron multistrat:



```
net = nn.Sequential(nn.Linear(5, 64),  
                    nn.ReLU(),  
                    nn.Linear(64, 128),  
                    nn.ReLU(),  
                    nn.Linear(128, 10))
```

- Acesta este un perceptron multistrat cu 2 straturi ascunse
- Primul primește 5 caractere și învață 64 de caracteristici noi din ele
- Al doilea învață din cele 64 de caracteristici 128 de caracteristici noi

Cum învățăm mai bine?

Regularizare cu Weight Decay

- Weight Decay e doar un alt nume pentru regularizarea ridge
- Regularizarea ridge implică menținerea ponderilor rețelei la valori mai mici
- Un model care face overfitting are de obicei ponderi mari

```
[ ] wd = 0.001  
    optimizer = torch.optim.SGD(net.parameters(), weight_decay=wd, lr=lr)
```

- Optimizatorul reprezintă metoda prin care se actualizează ponderile
- Mai sus, actualizarea se face prin intermediul stochastic gradient descent

Regularizare cu Dropout

- Dropout reprezintă altă metodă prin care putem reduce overfitting-ul
- În timpul antrenării, aleatoriu o parte din neuroni sunt opriți, adică că nu se activează sau ieșirea lor e 0
- Ideea e că modelul poate deveni prea dependent de anumiți neuroni sau combinații specifice de neuroni
- Astfel că, o parte bună din neuroni s-ar putea să nu contribuie mai deloc la sarcina pe care vrem să o rezolvăm
- Oprind neuroni în timpul antrenării, forțăm modelul să se bazeze pe toți neuronii, contribuind la o generalizare mai bună

Regularizare cu Dropout

- Dropout reprezintă altă metodă prin care putem reduce overfitting-ul

```
▶ dropout1, dropout2 = 0.2, 0.3

net = nn.Sequential(nn.Flatten(),
                    nn.Linear(784, 256),
                    nn.ReLU(),
                    # Add a dropout layer after the first fully connected layer
                    nn.Dropout(dropout1),
                    nn.Linear(256, 256),
                    nn.ReLU(),
                    # Add a dropout layer after the second fully connected layer
                    nn.Dropout(dropout2),
                    nn.Linear(256, 10))
```

- 20% din neuronii din primul strat ascuns sunt opriți la antrenare
- 30% din neuronii din al doilea strat ascuns sunt opriți la antrenare
- Stratul se adaugă după funcția de activare

- Atenție! Dropout se realizează doar la antrenare, ca să antrenăm modelul să generalizeze mai bine. La inferență (când facem preziceri), acel strat se va dezactiva
- PyTorch realizează dezactivarea automat atunci când trecem modelul în evaluation mode: `net.eval()`

Inițializarea optimă a ponderilor

- Felul în care se realizează inițializarea ponderilor înainte ca modelul să învețe este foarte important
- Prin diferite inițializări se poate ajunge la o convergență a modelului către o soluție mai bună sau doar la o convergență mai rapidă către o soluție destul de bună

```
[ ] def init_normal(m):  
    if type(m) == nn.Linear:  
        nn.init.normal_(m.weight, mean=0, std=0.01)  
        nn.init.zeros_(m.bias)  
  
net.apply(init_normal)  
net[0].weight.data[0], net[0].bias.data[0]  
  
⇒ (tensor([-0.0229,  0.0040,  0.0015, -0.0036]), tensor(0.))
```

Inițializarea cu distribuție gaussiană

```
[ ] def init_constant(m):  
    if type(m) == nn.Linear:  
        nn.init.constant_(m.weight, 1)  
        nn.init.zeros_(m.bias)  
  
net.apply(init_constant)  
net[0].weight.data[0], net[0].bias.data[0]  
  
⇒ (tensor([1., 1., 1., 1.]), tensor(0.))
```

Inițializarea cu o constantă

```
[ ] def xavier(m):  
    if type(m) == nn.Linear:  
        nn.init.xavier_uniform_(m.weight)  
  
net[0].apply(xavier)  
print(net[0].weight.data[0])  
  
⇒ tensor([-0.4192, -0.4082, -0.4657, -0.2263])
```

Inițializarea Xavier

(face ca semnalele să nu fie diminuat sau accentuat excesiv de la un strat la altul)

Realizarea de rețele
neuronale prin clase

Realizarea de rețele neuronale prin clase

- Putem crea clase care să reprezinte modelul nostru
- Acestea trebuie să extindă clasa `nn.Module`

```
[ ] class MLP(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.hidden = nn.Linear(20, 256) # Hidden layer  
        self.out = nn.Linear(256, 10) # Output layer  
  
    def forward(self, X):  
        return self.out(nn.ReLU()(self.hidden(X)))
```

- E foarte simplu: tot ce trebuie să facem este să inițializăm straturile în constructor, iar mai apoi să definim metoda forward, care spune cum ajungem de la intrare la ieșire prin acele straturi

That's it for today!
Now let's do some exercises.

That's it for today!

Now let's do some exercises.

Laboratory 8 – Exercises

1. Estimate the following linear model $y = Xw + b + \epsilon$, where $X \in \mathbb{R}^{2000 \times 3}$ consists of 2000 examples, each consisting of 3 features sampled from a standard normal distribution and ϵ obeys a normal distribution with mean 0 and standard deviation of 0.1. The true weights of the model will be $w = [0.15, -0.2, 0.1]^T$ and $b = 0.2$. Create the linear regression model (**do not use PyTorch's predefined layers**) and train it as follows:

- set the weights and bias to 3;
- set the batch size to 20;
- **implement** the mean squared error loss;
- **implement** the mini-batch gradient descent optimizer, $lr = 0.02$ (do not use the torch.optim package);
- set the total number of epochs to 40.

After the model is trained, compare the learned model parameters with the actual parameters.

2. Implement the softmax regression model in order to classify the images from the Imagenette dataset (10 classes) as follows:

- download the Imagenette train images;
- split the train images as follows: validation—4456, training—4456, and the rest of the images represents the testing dataset;
- set the batch size to 100;
- resize the images to (300, 100) resolution;
- initialize the weights at random with zero mean and standard deviation 0.01;
- use the cross-entropy loss function;
- use mini-batch gradient descent with a learning rate of 0.1 as the optimization algorithm;
- set the number of epochs to 5;

Evaluate the trained model on the test set.

3. Classify the Imagenette dataset (10 classes) using an MLP as follows:

- download the Imagenette train images;
- split the train images as follows: validation—5000, training—4000, and the rest of the images represents the testing dataset;
- set the batch size to 150;